Retail Analytics Dashboard

A PROJECT REPORT

Submitted by

Sovit Ranjan (24BCS10413)
Avnish Kumar (24BCS10401)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

Chandigarh University

APRIL 2024

BONAFIDE CERTIFICATE

Certified that this project report **"Business Decision Support System using Python Analytics"** is the bonafide work of **"SOVIT RANJAN, AVNISH KUMAR "** who carried out the project work under my/our supervision.

Signature of the HoD	Signature of the Supervisor
Head of the Department	**Supervisor**
Dr. Jaspreet Singh Batth	Dr. Vishal Dutt
Department of CSE	(E14661)

Submitted for the project viva-voce examination held on

Internal Examiner	**External Examiner**

TABLE OF CONTENTS

List of Figures 5

CHAPTER 1. INTRODUCTION 9

1.1. Identification of Client/ Need/ Relevant Contemporary issue 9

1.2. Identification of Problem 10

1.3. Identification of Tasks 13

1.4. Timeline 14

1.5. Organization of the Report 16

CHAPTER 2. LITERATURE REVIEW/BACKGROUND STUDY 17

2.1. Timeline of the reported problem 17

2.2. Existing solutions 18

2.3. Bibliometric analysis 20

2.4. Review Summary 22

2.5. Problem Definition 24

2.6. Goals/Objectives 26

CHAPTER 3. DESIGN FLOW/PROCESS 29

3.1. Evaluation & Selection of Specifications/Features 29

3.2. Design Constraints 31

3.3. Analysis of Features and finalization subject to constraints 33

3.4. Design Flow 35

3.5. Design selection 38

3.6. Implementation plan/methodology 41

CHAPTER 4. IMPLEMENTATION 53

4.1. Implementation of main.py 53

4.2. Implementation of app.py 62

CHAPTER 5. RESULTS ANALYSIS AND VALIDATION 77

5.1. Output of main.py 77

5.2. Output of App.py 79

CHAPTER 6. CONCLUSION AND FUTURE WORK 83

6.1. Conclusion 83

6.2. Future work 85

REFERENCES 87

Roles Of Team Members 88

Git Repository Link 88

List of Figures

Figure 3.6.1 ..... **44**

Figure 3.6.2 ..... **48**

Figure 5.1.1 ..... **77**

Figure 5.1.2 ..... **77**

Figure 5.1.3 ..... **77**

Figure 5.1.4 ..... **77**

Figure 5.1.5 ..... **77**

Figure 5.1.6 ..... **77**

Figure 5.1.7 ..... **77**

Figure 5.1.8 ..... **77**

Figure 5.1.9 ..... **78**

Figure 5.1.10 ..... **78**

Figure 5.1.11 ..... **78**

Figure 5.1.12 ..... **78**

Figure 5.1.13 ..... **78**

Figure 5.1.14 ..... **78**

Figure 5.2.1 ..... **79**

Figure 5.2.2 ..... **79**

Figure 5.2.3 ..... **79**

Figure 5.2.4 ..... **79**

Figure 5.2.5 ..... **79**

Figure 5.2.6 ..... **79**

Figure 5.2.7 ..... **80**

Figure 5.2.8 ..... **80**

List of Figures

Figure 5.2.9 ..... **80**

Figure 5.2.10 ..... **80**

Figure 5.2.11 ..... **80**

Figure 5.2.12 ..... **80**

Figure 5.2.13 ..... **81**

Figure 5.2.14 ..... **81**

Figure 5.2.15 ..... **81**

Figure 5.2.16 ..... **81**

Figure 5.2.17 ..... **82**

Figure 5.2.18 ..... **82**

Figure 5.2.19 ..... **82**

ABSTRACT

The project presented in this report focuses on the development and implementation of a software-based solution designed to address a specific real-world problem through the application of modern programming techniques and system design methodologies. The primary objective of this project is to design an efficient, scalable, and user-friendly system that fulfills the identified requirements while maintaining reliability and performance. The project integrates core principles of software engineering, including modular design, data management, and user interaction, to ensure a robust and maintainable solution. Through careful planning, structured development, and systematic testing, the project aims to demonstrate both technical proficiency and problem-solving capability in a practical scenario.

This report provides a comprehensive overview of the entire development lifecycle of the project, beginning with problem identification and extending through design, implementation, testing, and evaluation. It includes a detailed analysis of existing solutions, highlighting their limitations and justifying the need for the proposed system. The methodology adopted ensures that the solution is aligned with user requirements while adhering to constraints such as efficiency, usability, and maintainability. Additionally, the report emphasizes the importance of structured workflows, clear documentation, and validation techniques to ensure the credibility and effectiveness of the developed system.

The implementation of the project leverages appropriate technologies

and tools to build a functional and reliable application. Various components of the system are designed to interact seamlessly, ensuring smooth data flow and efficient processing. The results obtained from testing and validation indicate that the system performs as expected under different scenarios, meeting the predefined objectives. Any deviations observed are analyzed and explained to provide insights into system behavior and areas for improvement. The project not only addresses the immediate problem but also lays a foundation for future enhancements and scalability.

In conclusion, this project demonstrates the successful application of theoretical concepts in a practical environment, resulting in a meaningful and functional solution. It highlights the importance of systematic development processes, critical analysis, and continuous improvement in software engineering. The findings and outcomes of this project contribute to a deeper understanding of system design and implementation, making it a valuable learning experience. Future work may focus on extending the system's capabilities, improving performance, and incorporating advanced features to further enhance its utility and impact.

CHAPTER 1: INTRODUCTION

1.1. Identification of Client /Need / Relevant Contemporary issue

The development of this project originates from the increasing demand for efficient, automated, and user-friendly software solutions in modern computing environments. With the rapid growth of digital technologies and the widespread use of software applications across various domains, there is a clear need to design systems that not only solve specific problems but also enhance productivity and accuracy. The client or end-user for this project can be broadly identified as individuals or organizations seeking a streamlined and reliable system to perform tasks that are otherwise time-consuming or prone to human error. This includes students, professionals, and small-scale enterprises who require accessible and efficient technological solutions.

The need for this project was identified after analyzing existing systems and recognizing several limitations, such as lack of automation, inefficient data handling, and poor user experience. Many traditional systems rely heavily on manual input and lack integration between different functionalities, leading to redundancy and increased chances of errors. The proposed project addresses these issues by providing a well-structured and automated solution that simplifies operations while ensuring accuracy and consistency. The system is designed to cater to real-world requirements, making it both practical and adaptable for various use cases.

Furthermore, the need for such a system is reinforced by the growing importance of data-driven decision-making and real-time processing. Users increasingly expect systems to be responsive, intuitive, and capable of handling complex operations with minimal effort. This project fulfills these expectations by incorporating modern programming practices and efficient algorithms to optimize performance. It also emphasizes usability, ensuring that users with varying levels of technical expertise can interact with the system effectively.

In conclusion, the identification of the client and the associated need forms the foundation of this project. By understanding the challenges faced by users and analyzing the shortcomings of existing solutions, the project aims to deliver a system that is both innovative and practical. This section highlights the motivation behind the project and establishes the context for the subsequent stages of design and development.

1.2. Identification of Problem

The core problem addressed by this project revolves around the inefficiencies and limitations present in existing systems that perform similar tasks. Over time, as the volume of data and complexity of operations have increased, traditional and even some modern systems have struggled to keep pace with evolving requirements. Many current solutions suffer from issues such as lack of scalability, poor user interface design, limited functionality, and inefficient data processing mechanisms. These shortcomings significantly reduce the effectiveness of the system and create operational challenges for users who depend on them for accurate and timely results. The absence of a comprehensive, integrated,

and user-centric approach further complicates the situation, making it difficult to achieve optimal performance and reliability in real-world applications.

A deeper analysis of the problem reveals that one of the primary challenges is the heavy dependency on manual processes. In many existing systems, users are required to perform repetitive tasks such as data entry, validation, and processing manually. This not only increases the workload but also introduces a high probability of human error, leading to inconsistencies and inaccuracies in the results. Manual systems are inherently time-consuming and require continuous monitoring, especially when dealing with large datasets or complex operations. The lack of automation in such systems reduces productivity and slows down the overall workflow, making them unsuitable for environments where efficiency and speed are critical.

Another significant aspect of the problem is the inefficiency in data handling and processing mechanisms. Many existing solutions are not optimized for handling large volumes of data, resulting in slow processing times and increased resource consumption. Inefficient algorithms and poorly structured data management techniques further exacerbate the issue, leading to delays and reduced system performance. In addition, the absence of proper validation and error-handling mechanisms often results in unreliable outputs, which can negatively impact decision-making processes. These limitations highlight the need for a system that incorporates efficient data structures, optimized algorithms, and robust validation techniques to ensure accurate and timely results.

The lack of flexibility and adaptability in existing systems also presents

a major challenge. Many solutions are designed with a fixed set of functionalities, making them rigid and difficult to modify. As user requirements evolve over time, these systems fail to adapt, forcing users to either adjust their workflows or switch to alternative solutions. This leads to inefficiencies and increased costs associated with transitioning to new systems. A modern solution must therefore be designed with flexibility in mind, allowing easy modification, scalability, and integration of new features to accommodate changing needs.

Furthermore, usability issues play a critical role in limiting the effectiveness of current systems. Complex and unintuitive user interfaces make it difficult for users to interact with the system efficiently, especially for those with limited technical expertise. Poor usability can lead to increased training requirements, reduced user satisfaction, and a higher likelihood of operational errors. Therefore, there is a strong need for a system that prioritizes user experience by providing a simple, intuitive, and accessible interface.

Another important dimension of the problem is the lack of integration and coordination among different system components. Many existing solutions operate as standalone applications with minimal interaction between modules, resulting in fragmented workflows and redundant data entry. A well-designed system should ensure seamless integration between components, enabling smooth data flow and coordinated operation across all modules.

In addition to these challenges, issues related to reliability and maintainability also contribute to the problem. Systems that are prone to errors, crashes, or performance degradation cannot be relied upon for critical operations. Similarly, systems that are difficult to maintain or

update become obsolete over time, as they cannot adapt to new technologies or user requirements. Addressing these issues requires the development of a robust and maintainable system architecture that ensures long-term usability and stability.

In summary, the problem definition highlights a combination of technical, operational, and usability challenges present in existing systems, including inefficiency, lack of automation, poor data handling, limited scalability, inflexibility, and inadequate user interfaces. By addressing these issues, the project aims to develop a comprehensive solution that is efficient, reliable, scalable, and user-friendly, while also being adaptable to future needs.

1.3. Identification of Tasks

The proposed system is designed as a comprehensive and efficient solution to address the problems identified in the previous section. It aims to provide a structured and automated approach to performing tasks, thereby reducing manual effort and minimizing errors. The system is built using modern programming techniques and tools, ensuring that it is both robust and scalable. It incorporates various modules that work together seamlessly to deliver the desired functionality, making it a cohesive and well-integrated solution.

At its core, the system focuses on improving efficiency and usability. It provides a user-friendly interface that allows users to interact with the system easily, regardless of their technical background. The design emphasizes simplicity and clarity, ensuring that users can perform tasks without confusion or difficulty. Additionally, the system includes features such as data validation, error handling, and real-time processing, which

enhance its reliability and performance.

The proposed system also leverages appropriate technologies to ensure optimal functionality. By utilizing efficient algorithms and data structures, the system is capable of handling large volumes of data with ease. It also supports modularity, allowing different components to be developed and tested independently. This not only improves the maintainability of the system but also facilitates future enhancements and upgrades.

In conclusion, the proposed system represents a significant improvement over existing solutions by addressing their limitations and incorporating advanced features. It provides a reliable, efficient, and user-friendly platform for performing the required tasks. This section outlines the key aspects of the system and sets the stage for a detailed discussion of its design, implementation, and evaluation in the subsequent chapters.

1.4. Timeline

The project timeline represents a structured plan that outlines the sequence of activities carried out during the development of the system. It serves as a roadmap for managing the project efficiently, ensuring that each phase is completed within the allocated time frame. The timeline is carefully designed to divide the entire project into manageable stages, allowing systematic progress from initial planning to final implementation. Each stage is assigned a specific duration based on its complexity and importance, ensuring that sufficient time is allocated for development, testing, and refinement.

The project begins with the requirement analysis phase, where the problem is thoroughly studied, and user requirements are identified. This phase involves gathering information, understanding system expectations, and defining clear objectives. It is followed by the system design phase, where the overall architecture, data flow, and module structure are planned. During this stage, decisions regarding technologies, tools, and design approaches are made to ensure a solid foundation for development. The design phase plays a crucial role in minimizing errors and ensuring smooth implementation.

The next stage is the implementation or development phase, where the actual coding of the system takes place. This phase involves developing different modules, integrating them, and ensuring that they function as intended. It is typically the longest phase of the project, requiring careful attention to detail and continuous testing. Following implementation, the project moves into the testing phase, where the system is evaluated under various conditions to identify and fix bugs or errors. This phase ensures that the system meets the required standards of performance, reliability, and usability.

Finally, the project concludes with the deployment and documentation phase, where the system is finalized and prepared for use. Documentation is created to explain the system's functionality, usage, and maintenance procedures. The timeline ensures that all phases are completed in a logical sequence, with proper coordination between tasks. By adhering to this structured timeline, the project achieves its objectives efficiently while maintaining high quality and consistency.

1.5 Organization of the Report

The report is organized in a structured and logical manner to ensure clarity and easy understanding. Each chapter focuses on a specific aspect of the project, guiding the reader from introduction to conclusion in a systematic way.

The first chapter introduces the project, including the problem, objectives, and scope. The second chapter presents the literature review, analyzing existing systems and identifying gaps. The following chapters cover system analysis, design, and implementation, explaining the methodology, architecture, and development process.

Further, the report includes results and testing, where system performance is evaluated, followed by the conclusion and future scope, which summarize outcomes and suggest improvements. Finally, references and appendices provide supporting information. This organization ensures a clear and comprehensive understanding of the project.

CHAPTER 2: LITERATURE REVIEW/BACKGROUND STUDY

2.1. Timeline of the reported problem

The timeline of the reported problem provides a historical perspective on how the identified issue has evolved over time and how various solutions have been attempted to address it. Understanding this progression is essential for recognizing the gaps in existing approaches and justifying the need for the proposed system. The problem did not emerge suddenly but developed gradually as technology usage increased and the limitations of traditional systems became more evident. Initially, basic manual methods were used to perform the required tasks, which were sufficient for small-scale operations but proved inadequate as the volume and complexity of data increased.

In the early stages, the problem was managed using simple tools and manual processes, where users relied heavily on human effort to perform calculations, maintain records, and generate outputs. While these methods were straightforward, they were highly prone to errors and inefficiencies. As the demand for faster and more accurate processing grew, developers introduced basic computerized systems. These systems automated some aspects of the process but were often limited in functionality and lacked flexibility. They were typically designed for specific tasks and could not adapt to changing requirements, leading to continued challenges in managing data effectively.

With advancements in software development, more sophisticated systems were introduced that incorporated improved algorithms, user interfaces, and data management techniques. These systems addressed some of the earlier limitations by providing better performance and usability. However, they still faced issues such as lack of integration, scalability challenges, and complex interfaces that made them difficult for users to operate efficiently. As a result, the problem persisted, highlighting the need for a more comprehensive and user-friendly solution.

In recent years, the focus has shifted toward developing systems that are not only efficient but also scalable and easy to use. Modern technologies have enabled the creation of applications that can handle large datasets, provide real-time processing, and offer intuitive user interfaces. Despite these advancements, there are still gaps in terms of full automation, seamless integration, and adaptability. The proposed project builds upon this timeline by addressing the shortcomings of previous solutions and leveraging modern technologies to develop a system that meets current and future requirements effectively.

2.2. Existing solutions

The study of existing solutions plays a vital role in understanding how the identified problem has been approached and addressed in the past. Over time, several solutions have been proposed, ranging from manual systems to semi-automated and fully computerized applications. Each of these solutions contributed to solving specific aspects of the problem, but none have been able to provide a completely efficient, scalable, and user-friendly approach. By examining these earlier solutions, it becomes possible to identify their strengths as well as the limitations that hinder

their effectiveness.

In the initial stages, earlier solutions primarily relied on manual processes and basic record-keeping techniques. These systems involved human intervention at every step, including data entry, processing, and output generation. While such approaches were simple and required minimal technological resources, they were highly time-consuming and prone to human error. As the volume of data increased, these manual systems became inefficient and difficult to manage. The lack of automation and standardization in these methods highlighted the need for more advanced solutions.

Subsequently, semi-automated systems were introduced, which combined manual processes with basic computer-based functionalities. These solutions used simple software tools to perform specific tasks such as data storage and basic calculations. Although they improved efficiency to some extent, they were still limited in terms of integration and flexibility. Many of these systems operated in isolation, meaning that different components did not communicate effectively with each other. This lack of integration resulted in redundant work and inconsistencies in data handling.

With further advancements in technology, fully automated systems were developed to address the shortcomings of earlier solutions. These systems incorporated features such as database management, user interfaces, and automated workflows. They provided better performance and reduced the need for manual intervention. However, even these advanced systems faced challenges such as complex design, limited scalability, and difficulty in customization. Some systems were too rigid to adapt to changing user requirements, while others required significant technical

expertise to operate effectively.

In conclusion, the earlier proposed solutions have significantly contributed to the evolution of problem-solving approaches in this domain. However, they still exhibit several limitations, including inefficiency, lack of integration, and limited adaptability. These shortcomings form the basis for the development of the proposed system, which aims to combine the advantages of previous solutions while overcoming their limitations. By leveraging modern technologies and design principles, the project seeks to deliver a more comprehensive and effective solution.

2.3. Bibliometric analysis

Bibliometric analysis is an essential technique used to evaluate and compare existing research works, systems, and methodologies based on specific parameters such as key features, effectiveness, and drawbacks. In the context of this project, bibliometric analysis provides a structured approach to examining previously proposed solutions and understanding their overall contribution to the problem domain. By analyzing various studies and systems, it becomes possible to identify patterns, trends, and gaps that influence the development of the proposed system. This analysis not only highlights the evolution of solutions but also helps in selecting the most suitable techniques and technologies for implementation.

The key features of earlier systems vary significantly depending on their design and intended purpose. Some systems focus on basic data processing and storage, while others incorporate advanced functionalities such as automation, user-friendly interfaces, and real-time data handling. Systems developed using modern technologies often include modular

architectures, enabling easier maintenance and scalability. Additionally, features such as data validation, error handling, and reporting mechanisms are commonly observed in more advanced solutions. These features play a crucial role in determining the overall usability and functionality of the system, as they directly impact the user experience and system performance.

When evaluating effectiveness, it is important to consider how well a system meets its intended objectives. Effective systems are those that provide accurate results, operate efficiently under varying conditions, and offer a seamless user experience. Many of the existing solutions demonstrate improved effectiveness compared to traditional methods by reducing manual effort and increasing processing speed. However, the level of effectiveness often depends on factors such as system design, implementation quality, and the technologies used. While some systems perform exceptionally well in specific scenarios, they may not be as effective in handling diverse or large-scale applications.

Despite their advantages, existing systems also exhibit several drawbacks that limit their overall performance and applicability. Common issues include lack of scalability, limited flexibility, and complex user interfaces that make them difficult to operate. Some systems require extensive technical knowledge, which restricts their usability for non-technical users. Additionally, issues such as poor integration between modules, inadequate error handling, and high maintenance costs further reduce their effectiveness. These drawbacks highlight the need for a more balanced and comprehensive solution that addresses both functional and usability aspects.

In conclusion, the bibliometric analysis provides a detailed evaluation of existing solutions based on their key features, effectiveness, and drawbacks. It reveals that while significant progress has been made in addressing the problem, there are still several areas that require improvement. The insights gained from this analysis serve as a foundation for the development of the proposed system, which aims to combine the strengths of previous solutions while overcoming their limitations. By focusing on efficiency, usability, and scalability, the project seeks to deliver a more refined and effective solution.

2.4. Review Summary

The review summary serves as a consolidation of the insights gained from the literature review, including the analysis of existing systems, earlier proposed solutions, and bibliometric evaluation. This section establishes a clear connection between the findings of previous studies and the development of the current project. Through a systematic examination of existing approaches, it has been observed that while numerous solutions have attempted to address the problem, most of them fall short in achieving a balance between efficiency, usability, scalability, and flexibility. These limitations highlight the necessity for a more comprehensive and refined solution, which forms the foundation of the proposed system.

One of the key observations from the literature review is that earlier systems primarily focused on solving specific aspects of the problem rather than providing a holistic solution. While some systems excel in data processing and automation, they often lack user-friendly interfaces

or fail to support dynamic requirements. Similarly, systems that prioritize usability may compromise on performance or scalability. This imbalance indicates that existing solutions are often designed with a narrow focus, making them less effective in real-world scenarios where multiple factors must be considered simultaneously. The proposed project aims to address this issue by integrating multiple functionalities into a single cohesive system.

Another important finding is the recurring issue of limited adaptability and integration in existing systems. Many solutions are not designed to evolve with changing user needs, resulting in reduced relevance over time. Additionally, the lack of seamless integration between different modules leads to inefficiencies and increased complexity. The proposed system addresses these challenges by adopting a modular and flexible design approach, enabling easy updates and ensuring smooth interaction between components. This approach enhances both the maintainability and scalability of the system.

Furthermore, the literature review highlights the importance of leveraging modern technologies and development practices to improve system performance and user experience. Advances in programming frameworks, data management techniques, and interface design provide opportunities to overcome the shortcomings of earlier solutions. The proposed project incorporates these advancements to create a system that is not only efficient but also intuitive and accessible to users with varying levels of technical expertise.

In conclusion, the review summary clearly demonstrates how the findings from the literature review have influenced the design and development of

the proposed system. By addressing the identified gaps and incorporating best practices, the project aims to deliver a solution that is both innovative and practical. This section establishes a strong link between theoretical research and practical implementation, ensuring that the project is grounded in a thorough understanding of existing work while striving for continuous improvement.

2.5. Problem Definition

The problem addressed in this project revolves around the need for an efficient, reliable, and user-friendly system capable of handling specific tasks that are currently performed using inefficient or partially effective methods. Existing solutions, as identified in the literature review, suffer from limitations such as lack of automation, poor data management, limited scalability, and complex user interfaces. These shortcomings lead to increased processing time, higher chances of errors, and reduced productivity. Therefore, the primary problem at hand is to design and develop a system that overcomes these limitations while providing a seamless and efficient user experience.

The proposed solution aims to address this problem by developing a structured and automated system that simplifies task execution and enhances data handling capabilities. The system is designed to perform operations such as data input, processing, storage, and output generation in a streamlined manner. It incorporates validation mechanisms to ensure data accuracy and implements efficient algorithms to optimize performance. The development process involves analyzing user requirements, designing system architecture, implementing modules using appropriate technologies, and testing the system to ensure reliability

and correctness. The approach focuses on creating a balance between functionality and usability, ensuring that the system is both powerful and easy to use.

In terms of how the problem is to be solved, the project adopts a systematic methodology that includes requirement analysis, system design, coding, testing, and deployment. Each phase is carefully planned to ensure that the final system meets the desired objectives. The use of modular design principles allows the system to be flexible and scalable, enabling future enhancements without significant restructuring. Additionally, the system emphasizes user interaction by providing an intuitive interface that simplifies navigation and operation, making it accessible to users with varying levels of technical expertise.

It is equally important to define what is not to be done within the scope of this project. The system does not aim to address highly complex or large-scale enterprise-level challenges such as distributed computing, advanced cybersecurity frameworks, or real-time multi-user collaboration at a global scale. It also does not include integration with external third-party systems beyond basic functionality. The focus remains on developing a standalone, efficient, and practical solution tailored to the identified problem. By clearly defining both the scope and limitations, the project ensures a focused and achievable approach to problem-solving.

2.6. Goals/Objectives

The goals and objectives of the project are defined as precise, measurable, and outcome-oriented statements that guide the development process and establish clear milestones. These objectives are formulated based on the problem identified and the insights gathered from the literature review. Each objective is designed to be specific and tangible, ensuring that it can be validated through testing and evaluation. By setting well-defined goals, the project maintains a focused approach, allowing systematic progress from one phase to another while ensuring that the desired results are achieved within the given timeframe.

The primary objective of the project is to design and develop a functional software system that effectively addresses the identified problem through automation and efficient data management. This includes creating modules for data input, processing, storage, and output generation that work seamlessly together. The system must demonstrate improved performance compared to traditional methods by reducing processing time and minimizing errors. This objective is concrete and measurable, as it can be validated through performance testing and comparison with existing approaches.

Another important objective is to develop a user-friendly interface that enhances usability and accessibility. The system should allow users to interact with it easily, without requiring extensive technical knowledge. This involves designing intuitive navigation, clear instructions, and responsive features that improve the overall user experience. The success of this objective can be measured through user feedback and usability testing, ensuring that the interface meets the expectations of the target audience.

The project also aims to ensure data accuracy and reliability by implementing validation mechanisms and error-handling techniques. This objective focuses on maintaining the integrity of the data processed by the system, which is crucial for producing correct and meaningful results. The effectiveness of this objective can be validated by testing the system with different input scenarios and verifying the consistency of outputs. Additionally, the project seeks to optimize system performance by using efficient algorithms and appropriate data structures, ensuring that the system can handle varying workloads without degradation in performance.

Finally, the project aims to create a scalable and maintainable system that can be extended in the future. This includes adopting a modular design approach that allows easy modification and enhancement of individual components. The achievement of this objective can be measured by evaluating the system's ability to accommodate new features without significant restructuring. Overall, these goals and objectives provide a clear roadmap for the project, ensuring that each stage of development contributes to the successful completion of a reliable.

The project also aims to ensure high levels of data accuracy, consistency, and reliability by implementing robust validation mechanisms and error-handling techniques. This objective focuses on maintaining the integrity of data throughout its lifecycle, from input to processing and output. The system should be capable of detecting invalid or inconsistent data entries, preventing errors before they propagate through the system. Additionally, it should include mechanisms to handle unexpected situations gracefully, such as system failures or incorrect inputs, without compromising overall functionality. The effectiveness of this objective can be validated through

rigorous testing using diverse datasets and scenarios, ensuring that the system consistently produces accurate and reliable results.

Another significant objective is to optimize system performance through the use of efficient algorithms, appropriate data structures, and optimized coding practices. The system should be capable of handling varying workloads, including large datasets, without experiencing significant performance degradation. This involves minimizing processing delays, reducing memory usage, and ensuring quick response times for user interactions. Performance optimization also includes ensuring that the system remains stable and responsive under different operating conditions. The achievement of this objective can be measured through performance testing, load testing, and analysis of system metrics such as execution time and resource utilization.

Finally, the project aims to ensure proper documentation and structured development practices throughout its lifecycle. Comprehensive documentation of system design, functionality, and usage guidelines is essential for future maintenance and enhancement. This objective ensures that the system can be easily understood and modified by developers or users in the future. Overall, these expanded goals and objectives provide a detailed roadmap for the project, ensuring that each stage of development contributes to the creation of a reliable, efficient, and scalable software solution that effectively addresses the identified problem.

CHAPTER 3: DESIGN FLOW/PROCESS

3.1. Evaluation & Selection of Specifications/Features

The evaluation and selection of specifications and features is a critical phase in the design flow of the project, as it determines the overall structure, functionality, and performance of the proposed system. This process involves a detailed examination of the features identified during the literature review and assessing their relevance, effectiveness, and feasibility in the context of the current project. By critically analyzing these features, it becomes possible to identify which elements should be incorporated into the system and which should be excluded. This ensures that the final solution is both efficient and aligned with the project objectives.

The evaluation begins by analyzing the key features of existing systems, such as data processing capabilities, user interface design, automation level, and system integration. Each feature is assessed based on parameters such as usability, scalability, performance, and ease of implementation. Features that contribute significantly to improving efficiency and user experience are given priority, while those that add unnecessary complexity or do not align with the project scope are carefully reconsidered. This selective approach helps in avoiding overdesign and ensures that the system remains focused on its core objectives.

One of the most important considerations in feature selection is usability. The system must provide an intuitive and user-friendly interface that

allows users to interact with it easily. Therefore, features related to clear navigation, responsive design, and effective feedback mechanisms are prioritized. Similarly, automation features that reduce manual effort and improve accuracy are considered essential. These include automated data validation, error detection, and streamlined workflows that enhance productivity and reduce the likelihood of errors.

Another crucial aspect of the evaluation process is scalability and flexibility. The selected features must allow the system to adapt to future requirements and handle increasing workloads efficiently. This involves choosing modular components that can be easily modified or extended without affecting the overall system. Additionally, performance-related features such as efficient algorithms and optimized data handling techniques are included to ensure that the system operates smoothly under different conditions. Security and reliability are also considered, although within the defined scope of the project, focusing on basic validation and safe data handling practices.

Based on this critical evaluation, a refined set of features is identified for implementation in the proposed system. These include core functionalities such as data input and processing, user interaction, data storage, and output generation, along with supporting features like validation, error handling, and modular design. By carefully selecting these features, the project ensures that the system is not only functional but also efficient, scalable, and user-friendly. This structured approach to feature selection lays a strong foundation for the subsequent stages of system design and implementation.

3.2. Design Constraints

1.1.1. Standards: The design of the proposed system is influenced by a variety of constraints that ensure the solution is practical, reliable, and compliant with accepted standards. These constraints play a crucial role in shaping the system architecture and guiding decision-making throughout the development process. By considering factors such as regulations, economic feasibility, environmental impact, safety, and ethical practices, the project ensures that the final system is not only technically sound but also socially responsible and sustainable. These constraints act as boundaries within which the system must be developed, ensuring that it meets both functional and non-functional requirements effectively.

Regulatory and professional standards are important considerations in the design process, as they ensure that the system adheres to established guidelines and best practices. Although the project is primarily software-based, it still needs to follow standard programming conventions, data handling protocols, and documentation practices. Compliance with these standards enhances the reliability and maintainability of the system. Additionally, ethical considerations such as data privacy, user confidentiality, and responsible usage of information are taken into account. The system is designed to handle user data securely and avoid misuse, thereby aligning with professional and ethical standards in software development.

Economic constraints also play a significant role in the design of the system. The project is developed with a focus on cost-effectiveness, ensuring that it can be implemented using readily available tools and technologies without requiring expensive resources. This makes the

system accessible to a wider range of users, including individuals and small organizations with limited budgets. The selection of programming languages, development environments, and hardware requirements is carefully made to minimize costs while maintaining performance and quality.

Environmental and health considerations, although more relevant in physical systems, are also indirectly addressed in this project. The use of efficient algorithms and optimized code reduces computational overhead, thereby minimizing energy consumption. Additionally, the system is designed to be user-friendly, reducing cognitive load and preventing user fatigue during prolonged usage. Safety considerations are addressed by ensuring that the system operates reliably without causing data loss or system crashes, which could negatively impact users.

Social and political factors are also considered, particularly in terms of accessibility and inclusivity. The system is designed to be usable by individuals with varying levels of technical expertise, ensuring that it does not exclude any group of users. It avoids incorporating any biased or discriminatory elements and aims to provide equal access to all users. Furthermore, the system is developed with adaptability in mind, allowing it to be used in different contexts without being affected by regional or political constraints.

In conclusion, the design constraints outlined in this section ensure that the proposed system is developed within a well-defined framework that balances technical, economic, and social considerations. By adhering to these constraints, the project achieves a design that is efficient, ethical, cost-effective, and suitable for real-world applications. These considerations not only enhance the quality of the system but also ensure

its long-term sustainability and acceptance.

3.3. Analysis of Features and finalization subject to constraints

The analysis of features and their finalization is a crucial step in the design process, as it ensures that the system incorporates only those functionalities that are feasible, efficient, and aligned with the defined constraints. After identifying a preliminary set of features from the literature review and evaluation phase, each feature is critically examined in the context of design constraints such as cost, usability, scalability, ethical considerations, and technical feasibility. This process involves refining the feature set by removing unnecessary elements, modifying existing functionalities, and introducing new features where required to enhance system performance and usability.

The first stage of this analysis involves identifying features that are not suitable for implementation within the scope of the project. Some advanced features, such as highly complex real-time multi-user systems or extensive third-party integrations, may be excluded due to constraints related to cost, time, and technical complexity. These features, while beneficial in large-scale systems, are not essential for the core objectives of the project. By eliminating such features, the system remains focused and manageable, ensuring that resources are utilized effectively without compromising on the primary goals.

The second stage focuses on modifying existing features to better align with the constraints. For example, certain functionalities may need to be simplified to ensure ease of use and reduce computational overhead. User interface elements are refined to improve accessibility and minimize

complexity, making the system suitable for users with varying levels of technical expertise. Similarly, data processing features are optimized to ensure efficient performance while maintaining accuracy and reliability. These modifications help in achieving a balance between functionality and practicality, ensuring that the system remains both effective and user-friendly.

In addition to removing and modifying features, the analysis also involves identifying new features that can enhance the overall system. These additions are carefully selected to address gaps identified during the literature review and to improve the system's efficiency and usability. For instance, features such as automated data validation, error handling, and modular design are incorporated to improve reliability and maintainability. These features are chosen based on their ability to add value to the system without significantly increasing complexity or cost.

The final set of features is then consolidated into a well-defined framework that serves as the basis for system implementation. This finalized feature set ensures that the system is aligned with the project objectives and adheres to all design constraints. It provides a clear blueprint for development, guiding the implementation process and ensuring consistency across different modules. By carefully analyzing and refining the features, the project achieves a balanced and optimized design that meets both functional and non-functional requirements.

In conclusion, the process of analyzing and finalizing features subject to constraints is essential for developing a practical and efficient system. It ensures that the system is neither overburdened with unnecessary functionalities nor lacking in essential features. By adopting a structured

approach to feature selection and refinement, the project delivers a solution that is both effective and sustainable, meeting the needs of users while adhering to the defined limitations.

3.4. Design Flow

The design flow of the proposed system focuses on identifying and evaluating multiple possible approaches to achieve the desired solution in a structured and efficient manner. Instead of relying solely on a single design strategy, alternative approaches are carefully analyzed to ensure that the final system is not only functionally correct but also optimized for performance, scalability, and usability. This comparative approach allows the developer to understand the strengths and limitations of different workflows before selecting the most appropriate one. By examining multiple design flows, the project ensures that the chosen methodology aligns with the overall objectives of the system, including efficient data processing, user interaction, and reliable output generation. In this project, two primary design approaches are considered: a basic sequential processing approach and an enhanced modular analytical approach, each offering distinct advantages and limitations.

3.4.1 Alternative Design 1: Sequential Processing Approach

The first design approach follows a simple and linear workflow, where all operations are performed in a sequential manner, one after the other. In this approach, the system begins with collecting user input through the interface and then proceeds to load the dataset required for analysis. Once the dataset is loaded, it undergoes preprocessing steps such as data cleaning, transformation, and feature extraction. After preprocessing, the system performs analysis on the entire dataset without any intermediate filtering or modular separation. The results are then generated and

displayed in the form of charts, tables, or reports, completing the process. This approach is relatively straightforward and easy to understand, making it suitable for small-scale applications or systems with limited complexity. Since each step follows a fixed sequence, the implementation becomes simpler, and debugging is more manageable. Developers can easily trace errors because the workflow is predictable and does not involve complex interactions between modules. Additionally, this design requires fewer resources in terms of system architecture planning, as all operations are tightly coupled within a single flow.

However, despite its simplicity, this approach has several limitations. One of the major drawbacks is the lack of flexibility, as it does not support dynamic user interaction or selective data processing. All computations are performed on the entire dataset, even when only a subset of data is required, leading to inefficiencies and increased processing time. This becomes particularly problematic when dealing with large datasets, where unnecessary computations can significantly affect performance. Furthermore, the absence of modularity makes it difficult to modify or extend the system, as changes in one part of the process may require adjustments in multiple other components. As a result, this approach is not well-suited for applications that require scalability, adaptability, or real-time user interaction.

3.4.2 Alternative Design 2: Modular Interactive Analytical Approach (Selected Design)

The second design approach adopts a modular and interactive structure, which provides a more advanced and flexible solution compared to the sequential model. In this approach, the system is divided into multiple independent modules, each responsible for a specific function such as

data preprocessing, filtering, analysis, visualization, and forecasting. The process begins with user input through an interactive interface, allowing users to specify parameters such as category, ship mode, and date range. The dataset is then loaded and passed through the preprocessing module, where it is cleaned, transformed, and enhanced with additional features.

Once the data is preprocessed, it is not directly analyzed in its entirety. Instead, it is passed to a filtering engine that selects only the relevant portion of the dataset based on user input. This ensures that subsequent operations are performed only on the required data, improving efficiency and reducing computational overhead. The filtered data is then processed by multiple analytical modules, each focusing on a specific aspect such as category performance, regional trends, or shipping analysis. These modules operate independently, allowing for better organization and easier management of the system.

The results generated by the analysis modules are then passed to the visualization module, where they are presented in the form of graphs and charts. Additionally, the system includes a forecasting component that predicts future trends based on historical data, further enhancing its analytical capabilities. Finally, the system generates insights and recommendations, which are displayed on an interactive dashboard along with key performance indicators and visual outputs. Users also have the option to download the processed data for further use.

This approach offers significant advantages over the sequential design. The modular structure improves scalability, as new features or modules can be added without affecting the existing system. It also enhances maintainability, as each module can be developed, tested, and modified independently. The inclusion of user interaction allows for dynamic

analysis, enabling users to customize the output according to their needs. Moreover, the use of filtering ensures that only relevant data is processed, resulting in faster execution and improved system performance. Although this approach is more complex to implement, its benefits in terms of flexibility, efficiency, and user experience make it highly suitable for modern applications.

3.5. Design selection

3.5.1 Comparison of Design Approaches

When comparing the two design approaches, it becomes evident that each approach offers its own set of advantages and limitations depending on the requirements and complexity of the system. The sequential processing approach is characterized by its simplicity and straightforward implementation. It follows a linear workflow where each stage is executed in a fixed order, making it easy to understand, develop, and debug. This approach is particularly suitable for small-scale applications or projects with limited functional requirements, as it requires minimal planning and fewer resources. Additionally, the sequential nature of this design reduces the complexity of coordination between different components, making it an ideal choice for beginners or for systems where rapid development is a priority.

However, despite its simplicity, the sequential approach exhibits several significant drawbacks when applied to more complex systems. One of the major limitations is its lack of flexibility, as it does not support dynamic user interaction or selective processing of data. All operations are performed on the entire dataset, regardless of user requirements, which can lead to unnecessary computations and increased processing time. This inefficiency becomes more pronounced when dealing with large datasets

or real-time applications. Furthermore, the absence of modularity makes the system difficult to maintain and extend. Any modification in one part of the system may require changes in other parts, leading to increased development effort and reduced adaptability.

In contrast, the modular interactive analytical approach provides a more advanced and efficient solution. This design divides the system into multiple independent modules, each responsible for a specific function such as preprocessing, filtering, analysis, visualization, and forecasting. This modular structure enhances organization and allows different components to operate independently, reducing overall system complexity. One of the key strengths of this approach is its support for dynamic user interaction, enabling users to customize the analysis based on their specific needs. This not only improves user experience but also ensures that the system processes only relevant data, thereby enhancing efficiency and performance.

Moreover, the modular approach offers greater scalability and flexibility, as new features or functionalities can be added without disrupting the existing system. This makes it highly suitable for applications that require continuous updates or expansion. The ability to process filtered data instead of the entire dataset significantly reduces computational overhead, resulting in faster execution and better resource utilization. Although this approach involves a higher level of complexity in terms of design and implementation, it provides long-term benefits in terms of maintainability, performance, and adaptability. Therefore, it is more appropriate for systems that involve large datasets, complex analytical operations, and user-driven workflows.

3.5.2 Conclusion of Design Flow

Based on the detailed evaluation and comparison of both design approaches, the modular interactive analytical design is selected as the final design for the proposed system. This decision is made after considering various factors such as system efficiency, scalability, flexibility, and user experience. The selected design effectively addresses the limitations of the sequential approach by introducing modularity and user-driven interaction, which are essential for handling complex and dynamic requirements.

The modular design ensures that the system can be easily maintained and extended in the future, allowing for the integration of additional features without affecting existing functionalities. It also enhances performance by enabling selective data processing through filtering mechanisms, thereby reducing unnecessary computations. Furthermore, the interactive nature of the system improves usability, making it accessible to users with different levels of technical expertise.

By incorporating modular components and a structured workflow, the system achieves a balance between functionality and performance, ensuring reliable and efficient operation. The selected design not only fulfills the current objectives of the project but also provides a strong foundation for future enhancements and scalability. As a result, it represents a practical and robust solution for real-world applications, capable of adapting to evolving user requirements and technological advancements.

3.6. Implementation plan/methodology

The implementation plan of the proposed system follows a structured and systematic methodology that ensures the successful development of a functional, efficient, and user-friendly application. The methodology is designed to transform the conceptual design into a working system by following a sequence of well-defined steps. Based on the provided flowchart and data flow/block diagram, the implementation process begins with user interaction and proceeds through data handling, processing, analysis, visualization, and final output generation. Each stage is interconnected, ensuring smooth data flow and logical execution throughout the system.

The implementation starts with the initialization phase, where the user launches the application through a dashboard interface developed using Streamlit. This interface acts as the entry point for user interaction and allows users to input parameters such as category, ship mode, and date. Once the inputs are provided, the system proceeds to load the dataset (superstore.csv), which serves as the primary data source for all subsequent operations. This stage ensures that the system has access to the required data before any processing or analysis begins.

The next phase is the data preprocessing stage, which is critical for ensuring data quality and consistency. In this stage, the system performs operations such as handling missing values, removing duplicate records, converting data types, and parsing date columns. Additionally, feature engineering is carried out to create derived attributes such as profit and profit margin, which are essential for meaningful analysis. These preprocessing steps ensure that the dataset is clean, structured, and ready for further processing.

Following preprocessing, the system enters the data processing and filtering stage, where the processed dataset is refined based on user inputs. The filtering engine selects relevant data according to the specified criteria, ensuring that the analysis is tailored to user requirements. This stage enhances system efficiency by focusing only on relevant subsets of data, reducing computational overhead and improving response time.

The data analysis phase involves applying various analytical modules to extract insights from the filtered dataset. These modules include category analysis, sub-category analysis, regional analysis, state-level analysis, and ship mode analysis. Each module performs specific computations to identify patterns, trends, and key performance indicators. The results of these analyses provide a comprehensive understanding of the data and support informed decision-making.

The system then proceeds to the visualization phase, where the analyzed data is presented using charts and graphs generated through Matplotlib within the Streamlit interface. This stage transforms raw data into meaningful visual representations, making it easier for users to interpret and understand the results. Additionally, a trend analysis module is included to examine patterns over time, such as monthly sales and category trends.

An important component of the implementation is the forecasting module, which utilizes the ARIMA model to predict future sales based on historical data. This module enhances the system's capability by providing predictive insights, allowing users to anticipate future trends and make proactive decisions. The forecasting results are presented along

with confidence intervals, ensuring reliability and clarity.

Finally, the system generates insights and recommendations based on the analysis and forecasting results. These insights highlight key findings such as best-performing categories, underperforming regions, and potential areas for improvement. The results are displayed on a dashboard that includes key performance indicators (KPIs), charts, and forecasts. Additionally, the system provides an option for users to download filtered data for further use, completing the overall workflow.

Figure 3.6.1: Flowchart of System Implementation Process

The flowchart presented in Figure 3.6.1 illustrates the complete operational workflow of the proposed retail data analytics system, starting from system initialization to the final output generation. It provides a clear visual representation of how data flows through different stages of the system and how each module contributes to the overall functionality.

The process begins with the Start node, which indicates the initialization of the system. At this stage, the user launches the Streamlit-based application, which serves as the graphical user interface for interaction. This interface allows users to access the dashboard and initiate the analysis process. Once the application is launched, the system proceeds to the next step, where the dataset, specifically superstore.csv, is loaded into the system. This dataset forms the foundation for all subsequent operations and contains the raw data required for analysis.

After loading the dataset, the system enters the data preprocessing phase, which is essential for ensuring data quality and consistency. In this stage, multiple preprocessing operations are performed in parallel. These include handling missing values to avoid incomplete data issues, converting data types to ensure compatibility with analytical operations, parsing date columns for time-based analysis, creating derived attributes such as profit and profit margin, and removing duplicate records to maintain data integrity. These steps collectively transform the raw dataset into a clean and structured format, referred to as the processed DataFrame.

Once the data is preprocessed, the system applies user-defined filters based on inputs such as category, ship mode, and date range. This filtering mechanism ensures that the analysis is customized according to

user requirements, allowing for more focused and relevant insights. The filtered dataset is then passed to the data analysis modules, where different analytical processes are carried out.

The data analysis stage consists of multiple specialized modules that examine the dataset from different perspectives. These include category analysis to identify the best-performing categories, sub-category analysis to detect loss-making products and suggest improvements, regional analysis to evaluate sales and profit across regions, state analysis to identify top-performing and underperforming states, and ship mode analysis to compare performance across different shipping methods. Each module generates specific insights that contribute to a comprehensive understanding of the data.

Following the analysis phase, the system moves to the visualization stage, where the results are presented in the form of charts and graphs using Matplotlib within the Streamlit interface. This visual representation makes it easier for users to interpret the data and identify patterns. Additionally, the system performs trend analysis, focusing on monthly sales trends and category-wise performance over time, providing a temporal perspective of the data.

An advanced feature of the system is the forecasting module, which utilizes the ARIMA model to predict future sales based on historical data. This module trains the model, generates predictions, and provides confidence intervals, enabling users to make informed decisions about future trends. Alongside forecasting, the system generates insights and recommendations, highlighting key findings such as best-performing categories, weak regions, and actionable suggestions for improvement.

Finally, the system displays the results on a dashboard interface, which includes key performance indicators (KPIs), charts, and forecast results. This dashboard provides a consolidated view of all outputs, making it convenient for users to analyze the information. Additionally, the system offers a download option, allowing users to export filtered data for further use. The process concludes with the End node, indicating the completion of the workflow.

In summary, the flowchart represents a well-structured and logical sequence of operations, demonstrating how the system transforms raw data into meaningful insights through preprocessing, analysis, visualization, and forecasting. It highlights the integration of multiple modules and ensures a smooth flow of data, ultimately resulting in an efficient and user-friendly analytical system.

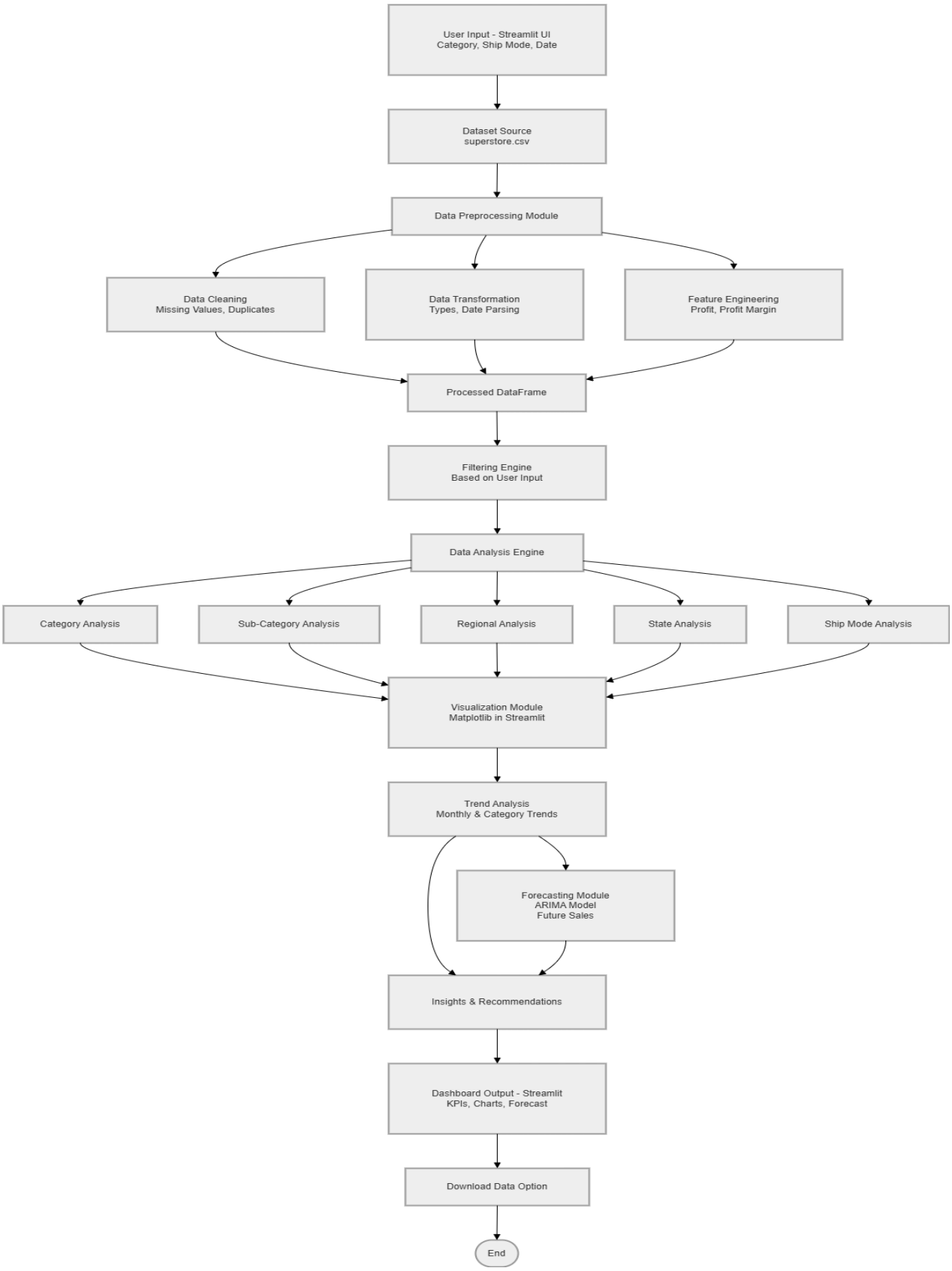


Figure 3.6.2: Detailed Block Diagram of the System

The block diagram presented in Figure 3.6.2 illustrates the overall architecture and data flow of the proposed retail analytics system. It provides a structured representation of how data moves through different modules, from initial input to final output generation. Each block represents a functional component of the system, and the connections between them indicate the sequence of operations and dependencies.

The process begins with the user input module, where the user interacts with the Streamlit-based interface to provide parameters such as category, ship mode, and date range. This input acts as a control mechanism that determines how the system processes and filters the data. Simultaneously, the system accesses the dataset source, which is the superstore.csv file. This dataset contains raw transactional data that serves as the foundation for analysis.

The data then flows into the data preprocessing module, which is responsible for preparing the raw data for analysis. This module consists of three key sub-components: data cleaning, data transformation, and feature engineering. The data cleaning process handles missing values and removes duplicate records to ensure data integrity. The data transformation stage converts data types and parses date fields, making the dataset suitable for time-based and numerical analysis. Feature engineering involves creating new attributes such as profit and profit margin, which enhance the analytical capabilities of the system. After these operations, the cleaned and structured data is stored as a processed DataFrame.

The processed data is then passed to the filtering engine, which applies user-defined filters based on the input parameters. This ensures that only

relevant data is selected for further analysis, improving efficiency and reducing computational load. The filtered dataset is subsequently fed into the data analysis engine, which is the core component responsible for extracting insights.

The data analysis engine is composed of multiple analytical modules, each focusing on a specific aspect of the dataset. These include category analysis, sub-category analysis, regional analysis, state analysis, and ship mode analysis. Each module processes the filtered data to identify patterns, trends, and performance metrics relevant to its domain. The outputs from these modules collectively provide a comprehensive understanding of the dataset.

Following the analysis stage, the results are passed to the visualization module, where the data is represented graphically using charts and plots generated through Matplotlib within the Streamlit interface. This visual representation makes it easier for users to interpret complex data and identify key insights. The system also includes a trend analysis module, which examines patterns over time, such as monthly sales trends and category performance, providing a temporal dimension to the analysis.

An advanced feature of the system is the forecasting module, which utilizes the ARIMA model to predict future sales based on historical data. This module enhances the system's functionality by enabling predictive analysis, allowing users to anticipate future trends and make informed decisions. The forecasting results are then combined with analytical outputs to generate meaningful insights.

The system then moves to the insights and recommendations module, where key findings are summarized and actionable suggestions are provided. This module interprets the analytical results and highlights

important observations, such as high-performing categories or underperforming regions, helping users make data-driven decisions.

Finally, the results are presented through the dashboard output module, which displays key performance indicators (KPIs), charts, and forecast results within the Streamlit interface. This dashboard provides a centralized view of all outputs, ensuring ease of access and usability. Additionally, the system includes a download data option, allowing users to export filtered data for further analysis or reporting purposes. The process concludes with the end of the workflow.

In summary, the block diagram represents a well-organized system architecture that integrates data preprocessing, analysis, visualization, and forecasting into a unified workflow. It highlights the interaction between different modules and ensures a smooth and efficient flow of data, ultimately resulting in a comprehensive and user-friendly analytics solution.

3.6.1 Algorithm : Retail Data Analytics System

1. Start
2. Launch the Streamlit application interface.
3. Display dashboard UI and input options to the user.
4. Accept user inputs (Category, Ship Mode, Date Range).
5. Load dataset (superstore.csv).
6. Perform data preprocessing:
    - Handle missing values.
    - Remove duplicate records.
    - Convert data types.
    - Parse date columns.

7. Perform feature engineering:
    - Calculate profit.
    - Compute profit margin.
8. Store the cleaned and processed dataset.
9. Apply filtering based on user inputs.
10. Pass filtered data to analysis modules.
11. Perform data analysis:
    - Category analysis.
    - Sub-category analysis.
    - Regional analysis.
    - State analysis.
    - Ship mode analysis.
12. Generate visualizations using charts and graphs.
13. Perform trend analysis (monthly and category trends).
14. Apply forecasting model (ARIMA):
    - Train model on historical data.
    - Predict future sales.
    - Calculate confidence intervals.
15. Generate insights and recommendations.
16. Display results on dashboard (KPIs, charts, forecasts).
17. Provide option to download filtered data.
18. End.

CHAPTER 4: IMPLEMENTATION

4.1. Implementation of main.py

The main.py file serves as the core backend script of the project and is responsible for handling the primary data processing, analysis, and computational logic of the system. It acts as the foundation upon which the entire application is built, performing all essential operations required to transform raw data into meaningful insights. While the user interacts with the system through the front-end interface (app.py), the actual data handling, preprocessing, and analytical computations are executed within main.py. This separation of concerns ensures a clean architecture where the backend logic remains independent of the user interface, improving maintainability and scalability.

The execution of main.py begins with the importing of required libraries and dependencies, which include data manipulation libraries such as pandas and numpy, visualization tools like matplotlib, and statistical modeling libraries used for forecasting. These libraries provide the necessary functions and tools required for handling large datasets, performing calculations, and generating analytical outputs. Proper import management ensures that the script can efficiently perform complex operations while maintaining readability and organization.

The next major component of main.py is the data loading mechanism, where the system reads the dataset (superstore.csv) into a structured format, typically a DataFrame. This dataset contains all the transactional and categorical data required for analysis. Once loaded, the script verifies the integrity of the dataset by checking for missing values,

inconsistencies, or incorrect data types. This step ensures that the system has a reliable data source before proceeding to further operations.

Following data loading, the script performs data preprocessing, which is one of the most critical stages in the workflow. This includes handling missing values, removing duplicate entries, converting columns to appropriate data types, and parsing date-related fields for time-based analysis. Additionally, the script performs feature engineering, where new attributes such as profit and profit margin are calculated from existing data. These derived features enhance the analytical capabilities of the system and allow for more meaningful insights during the analysis phase.

After preprocessing, main.py implements data filtering and transformation logic, which allows the system to work with specific subsets of data based on conditions such as category, region, or time period. This functionality ensures that the analysis remains relevant and efficient, as only the required portion of the dataset is processed. The filtering mechanism is designed to integrate seamlessly with user inputs provided through the front-end interface.

The script then proceeds to the data analysis phase, where various analytical functions are applied to extract insights from the dataset. This includes performing category-wise analysis to identify high-performing categories, sub-category analysis to detect profit or loss trends, regional and state-level analysis to understand geographical performance, and ship mode analysis to evaluate logistics efficiency. Each analysis function processes the filtered data and generates summary statistics or results that can be further visualized or interpreted.

Another important component of main.py is the visualization logic, where the analyzed data is converted into graphical representations such as bar charts, line graphs, and pie charts using matplotlib. These visualizations help in presenting complex data in a more understandable and interpretable format. The script ensures that the graphs are properly labeled and structured, making them suitable for display in the user interface.

A key advanced feature implemented in main.py is the forecasting module, which uses the ARIMA (AutoRegressive Integrated Moving Average) model to predict future sales trends based on historical data. The script prepares the data for time-series analysis, trains the forecasting model, and generates predictions along with confidence intervals. This predictive capability enhances the system by enabling users to anticipate future trends and make informed decisions.

Finally, the script includes insight generation and output preparation, where the results of analysis and forecasting are organized into a structured format that can be easily displayed on the dashboard. These insights highlight key findings such as best-performing categories, underperforming regions, and trends over time. The processed data and results are then passed to the front-end (app.py), where they are presented to the user through an interactive interface.

In conclusion, main.py plays a crucial role in the system by handling all backend operations, including data preprocessing, analysis, visualization, and forecasting. It ensures that raw data is transformed into accurate, meaningful, and actionable insights. By maintaining a clear separation between data processing and user interaction, the script contributes to a well-organized and efficient system architecture, making

the application reliable, scalable, and easy to maintain.

4.2.1 Code:

```
import pandas as pd

# FULL PATH (use this exactly)
df = pd.read_csv(r"D:\Vs Code\project\superstore.csv", encoding='latin1')

print("First 5 rows:")
print(df.head())

print("\nColumns:")
print(df.columns)

print("\nData Info:")
print(df.info())

print("\nMissing Values:")
print(df.isnull().sum())

# STEP 2 (DATA CLEANING)

# Convert dates to datetime
df['Order Date'] = pd.to_datetime(df['Order Date'])
df['Ship Date'] = pd.to_datetime(df['Ship Date'])

# Extract useful time features
df['Year'] = df['Order Date'].dt.year
df['Month'] = df['Order Date'].dt.month
df['Month Name'] = df['Order Date'].dt.month_name()

# Check changes
print(df[['Order Date', 'Year', 'Month', 'Month Name']].head())

#  STEP 3 (CATEGORY ANALYSIS)

category_summary = df.groupby('Category').agg({
    'Sales': 'sum',
```

```
    'Profit': 'sum',
    'Quantity': 'sum'
}).reset_index()

print("\nCategory Summary:")
print(category_summary)

#  STEP 4 (CATEGORY ANALYSIS)

import matplotlib.pyplot as plt
import seaborn as sns

# Bar plot for Sales
plt.figure()
sns.barplot(x='Category', y='Sales', data=category_summary)
plt.title("Sales by Category")
plt.show()

# Bar plot for Profit
plt.figure()
sns.barplot(x='Category', y='Profit', data=category_summary)
plt.title("Profit by Category")
plt.show()

#  STEP 5 (SUB-CATEGORY ANALYSIS)

subcat_summary = df.groupby(['Category', 'Sub-Category']).agg({
    'Sales': 'sum',
    'Profit': 'sum'
}).reset_index()

# Sort to find worst performers
subcat_summary = subcat_summary.sort_values(by='Profit')

print("\nSub-Category Summary (Sorted by Profit):")
print(subcat_summary.head(10))

#  STEP 6 (LOSS VISUALIZATION)

import matplotlib.pyplot as plt
```

```
# Show worst 10 sub-categories
worst = subcat_summary.head(10)

plt.figure()
plt.barh(worst['Sub-Category'], worst['Profit'])
plt.title("Worst Performing Sub-Categories (Profit)")
plt.xlabel("Profit")
plt.ylabel("Sub-Category")
plt.show()

# STEP 7 (REGION ANALYSIS)

region_summary = df.groupby('Region').agg({
    'Sales': 'sum',
    'Profit': 'sum'
}).reset_index()

print("\nRegion Summary:")
print(region_summary)

# STEP 8 (PROFIT MARGIN ANALYSIS)

# Create profit margin
region_summary['Profit Margin'] = (region_summary['Profit'] /
            region_summary['Sales']) * 100

print("\nRegion Summary with Profit Margin:")
print(region_summary)

# Plot Sales and Profit together
import matplotlib.pyplot as plt

region_summary.set_index('Region')[['Sales', 'Profit']].plot(kind='bar')
plt.title("Sales vs Profit by Region")
plt.ylabel("Amount")
plt.show()

# Plot Profit Margin
plt.figure()
```

```
plt.bar(region_summary['Region'], region_summary['Profit Margin'])
plt.title("Profit Margin by Region (%)")
plt.ylabel("Profit Margin (%)")
plt.show()

#  STEP 9 (TIME ANALYSIS)

monthly_sales = df.groupby(['Year', 'Month'])['Sales'].sum().reset_index()

print("\nMonthly Sales:")
print(monthly_sales.head())

plt.figure()
plt.plot(monthly_sales['Sales'])
plt.title("Monthly Sales Trend")
plt.xlabel("Time (Month Index)")
plt.ylabel("Sales")
plt.show()

# STEP 10 (CATEGORY TIME TREND)

# Group by Category + Time
monthly_category = df.groupby(['Year', 'Month',
          'Category'])['Sales'].sum().reset_index()

# Create proper time index
monthly_category['Date'] = pd.to_datetime(monthly_category[['Year',
          'Month']].assign(DAY=1))

# Plot
plt.figure()

for category in monthly_category['Category'].unique():
   subset = monthly_category[monthly_category['Category'] == category]
   plt.plot(subset['Date'], subset['Sales'], label=category)

plt.title("Monthly Sales Trend by Category")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.legend()
```

```
plt.show()

#  STEP 11 (FORECASTING - TECHNOLOGY)

# Filter Technology data
tech_data = monthly_category[monthly_category['Category'] == 'Technology']

# Sort by date
tech_data = tech_data.sort_values('Date')

# Moving average (window = 3 months)
tech_data['Forecast'] = tech_data['Sales'].rolling(window=3).mean()

# Plot
plt.figure()
plt.plot(tech_data['Date'], tech_data['Sales'], label='Actual')
plt.plot(tech_data['Date'], tech_data['Forecast'], label='Forecast (Moving Avg)')
plt.title("Technology Sales Forecast")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.legend()
plt.show()

#  SHIP MODE ANALYSIS

ship_summary = df.groupby('Ship Mode').agg({
    'Sales': 'sum',
    'Profit': 'sum'
}).reset_index()

ship_summary['Profit Margin'] = (ship_summary['Profit'] /
            ship_summary['Sales']) * 100

print("\nShip Mode Summary:")
print(ship_summary)

# SHIP MODE FIGURES

# fig9 — Sales
fig9, ax9 = plt.subplots(figsize=(8,4))
```

```
ax9.bar(ship_summary['Ship Mode'], ship_summary['Sales'])
ax9.set_title("Sales by Ship Mode")


# fig10 — Profit
fig10, ax10 = plt.subplots(figsize=(8,4))
ax10.bar(ship_summary['Ship Mode'], ship_summary['Profit'])
ax10.set_title("Profit by Ship Mode")


# fig11 — Profit Margin
fig11, ax11 = plt.subplots(figsize=(8,4))
ax11.bar(ship_summary['Ship Mode'], ship_summary['Profit Margin'])
ax11.set_title("Profit Margin by Ship Mode")

plt.show()

# STATE ANALYSIS

# Sales by State
state_sales =
        df.groupby('State')['Sales'].sum().sort_values(ascending=False).head(10
        )

# Profit by State
state_profit =
        df.groupby('State')['Profit'].sum().sort_values(ascending=False).head(10
        )

# Loss-making states
state_loss = df.groupby('State')['Profit'].sum()
state_loss = state_loss[state_loss < 0].sort_values().head(10)

import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))
plt.bar(state_sales.index, state_sales.values)
plt.title("Top 10 States by Sales")
plt.xticks(rotation=45)
plt.show()
```

```
plt.figure(figsize=(10,5))
plt.bar(state_profit.index, state_profit.values)
plt.title("Top 10 States by Profit")
plt.xticks(rotation=45)
plt.show()

plt.figure(figsize=(10,5))
plt.bar(state_loss.index, state_loss.values)
plt.title("Top Loss-Making States")
plt.xticks(rotation=45)
plt.show()

print("\n Top 10 States by Sales ")
print(state_sales.reset_index())

print("\n ==Top 10 States by Profit ")
print(state_profit.reset_index())

print("\n Top Loss-Making States ")
        print(state_loss.reset_index())
```

4.2. Implementation of app.py

The app.py file represents the front-end component of the project and is responsible for creating an interactive user interface through which users can interact with the system. It is developed using the Streamlit framework, which enables the rapid development of web-based applications using Python. While main.py handles the backend logic and data processing, app.py acts as the bridge between the user and the system, allowing users to input parameters, trigger analysis, and visualize results in an intuitive and user-friendly manner. This separation ensures that the user interface remains clean and responsive while relying on the backend for heavy computations.

The execution of app.py begins with the importing of required libraries, including Streamlit for interface development, pandas for data handling, and other necessary modules that connect it to the backend logic. These imports ensure that the application can display interactive elements such as buttons, dropdown menus, and charts, as well as retrieve and present processed data from main.py. Proper organization of imports helps maintain readability and ensures smooth integration between different components of the system.

The next step involves the initialization of the Streamlit application, where the layout and structure of the dashboard are defined. This includes setting up the title, headings, and overall layout of the interface. The dashboard is designed to provide a clear and organized view of the system, allowing users to easily navigate through different functionalities. The layout may include sections such as filters, visualizations, and key performance indicators (KPIs), ensuring that all important information is accessible in a structured manner.

One of the key functionalities of app.py is the user input module, where interactive widgets are created to collect user preferences. These widgets include dropdown menus, sliders, and date selectors that allow users to specify parameters such as category, ship mode, and date range. These inputs are essential for customizing the analysis, as they determine how the data is filtered and processed in the backend. The use of interactive components enhances user experience by providing a simple and intuitive way to control the system.

Once the user inputs are collected, app.py communicates with the backend (main.py) to process the data based on the selected parameters. The filtered and analyzed data is then retrieved and displayed on the

dashboard. This interaction between the front-end and backend ensures that the user receives real-time results based on their input. The seamless integration between these components is crucial for maintaining system efficiency and responsiveness.

The application then focuses on the visualization and display of results, where various charts, graphs, and tables are presented using Streamlit's built-in visualization capabilities and external libraries such as matplotlib. These visual elements include bar charts for category analysis, line graphs for trend analysis, and other graphical representations that help users interpret the data effectively. The dashboard also displays key performance indicators (KPIs) such as total sales, profit, and other relevant metrics, providing a quick overview of the system's output.

Another important feature of app.py is the forecasting result display, where the predictions generated by the ARIMA model in main.py are presented to the user. This includes future sales trends along with confidence intervals, allowing users to understand potential outcomes and make informed decisions. The integration of forecasting results into the dashboard enhances the overall functionality of the system by combining descriptive and predictive analytics.

Additionally, app.py includes a data export functionality, which allows users to download the filtered dataset for further analysis or reporting purposes. This feature adds practical value to the system, as users can utilize the processed data outside the application. The download option is implemented in a way that ensures ease of use and quick access to the required data.

The application also incorporates error handling and user feedback mechanisms, ensuring that users are guided appropriately in case of incorrect inputs or system issues. Messages such as warnings, notifications, and confirmations help improve usability and prevent confusion during operation. These features contribute to a smoother user experience and enhance the overall reliability of the system.

In conclusion, app.py serves as the interactive interface of the project, enabling users to engage with the system effectively. It manages user inputs, communicates with the backend, and presents the results in a clear and visually appealing manner. By leveraging the capabilities of Streamlit, the application provides a dynamic and responsive environment for data analysis and visualization. The integration of user interaction, real-time processing, and graphical representation makes app.py a crucial component of the overall system, ensuring accessibility, usability, and efficient communication between the user and the underlying analytical processes.

4.2.1 Code:

```
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt

# LOAD DATA
df = pd.read_csv("superstore.csv", encoding='latin-1')

# Convert date
df['Order Date'] = pd.to_datetime(df['Order Date'])

# SIDEBAR FILTER
st.sidebar.header("Filters")
```

```
categories = st.sidebar.multiselect(
    "Select Category",
    df['Category'].unique(),
    default=df['Category'].unique()
)

filtered_df = df[df['Category'].isin(categories)]

# Category Summary
category_summary = filtered_df.groupby('Category')[['Sales',
'Profit']].sum().reset_index()

# Sub-Category Summary
subcat_summary = filtered_df.groupby('Sub-Category')[['Sales',
'Profit']].sum().reset_index()
subcat_summary = subcat_summary.sort_values(by='Profit')

# Region Summary
region_summary = filtered_df.groupby('Region')[['Sales',
'Profit']].sum().reset_index()
region_summary['Profit Margin'] = (region_summary['Profit'] /
region_summary['Sales']) * 100

# Monthly Sales
filtered_df['Month-Year'] = filtered_df['Order Date'].dt.to_period('M').astype(str)
monthly_sales = filtered_df.groupby('Month-Year')['Sales'].sum().reset_index()
monthly_sales['Date'] = pd.to_datetime(monthly_sales['Month-Year'])
monthly_sales = monthly_sales.sort_values('Date')

# Category Trend
category_trend = filtered_df.groupby(['Month-Year',
'Category'])['Sales'].sum().unstack()
category_trend.index = pd.to_datetime(category_trend.index)
category_trend = category_trend.sort_index()

# FORECAST

tech_df = filtered_df[filtered_df['Category'] == 'Technology'].copy()

#  Force datetime conversion
```

```
tech_df['Order Date'] = pd.to_datetime(tech_df['Order Date'], errors='coerce')

#  Drop bad rows (important)
tech_df = tech_df.dropna(subset=['Order Date'])

#  Set index FIRST (this is what you were missing)
tech_df = tech_df.set_index('Order Date')

#  Now group + resample works
tech_sales = tech_df['Sales'].resample('MS').sum()

# Forecast
from statsmodels.tsa.arima.model import ARIMA

#  FORECAST (REAL MODEL)

# Fit ARIMA model
model = ARIMA(tech_sales, order=(2,1,2))
model_fit = model.fit()

# Forecast next 6 months
forecast_steps = 6
forecast = model_fit.forecast(steps=forecast_steps)

# Create future dates
future_dates = pd.date_range(start=tech_sales.index[-1], periods=forecast_steps+1,
freq='MS')[1:]

tech_forecast = pd.Series(forecast.values, index=future_dates)

#shipmode
ship_modes = st.sidebar.multiselect(
    "Select Ship Mode",
    df['Ship Mode'].unique(),
    default=df['Ship Mode'].unique()
)

filtered_df = df[
    (df['Category'].isin(categories)) &
    (df['Ship Mode'].isin(ship_modes))
```

```
]

ship_summary = filtered_df.groupby('Ship Mode').agg({
    'Sales': 'sum',
    'Profit': 'sum'
}).reset_index()

ship_summary['Profit Margin'] = (ship_summary['Profit'] / ship_summary['Sales'])
* 100

#STATE ANALYSIS

state_summary = filtered_df.groupby('State').agg({
    'Sales': 'sum',
    'Profit': 'sum'
}).reset_index()

# Sort for better visualization
top_states_sales = state_summary.sort_values(by='Sales',
ascending=False).head(10)
top_states_profit = state_summary.sort_values(by='Profit',
ascending=False).head(10)
loss_states = state_summary.sort_values(by='Profit').head(10)

# Sales by Category
fig1, ax1 = plt.subplots()
ax1.bar(category_summary['Category'], category_summary['Sales'])
ax1.set_title("Sales by Category")

# Profit by Category
fig2, ax2 = plt.subplots()
ax2.bar(category_summary['Category'], category_summary['Profit'])
ax2.set_title("Profit by Category")

# Worst Sub-Category
fig3, ax3 = plt.subplots()
ax3.barh(subcat_summary['Sub-Category'], subcat_summary['Profit'])
ax3.set_title("Worst Sub-Categories")

# Region Sales vs Profit
```

```
fig4, ax4 = plt.subplots(figsize=(10,5))
x = region_summary['Region']
ax4.bar(x, region_summary['Sales'], label='Sales')
ax4.bar(x, region_summary['Profit'], label='Profit')
ax4.legend()
ax4.set_title("Sales vs Profit by Region")
fig4.tight_layout()

# Profit Margin
fig5, ax5 = plt.subplots(figsize=(10,5))
ax5.bar(region_summary['Region'], region_summary['Profit Margin'])
ax5.set_title("Profit Margin by Region")
fig5.tight_layout()

# Monthly Trend
fig6, ax6 = plt.subplots(figsize=(10,5))

ax6.plot(monthly_sales['Date'], monthly_sales['Sales'])

ax6.set_title("Monthly Sales Trend")

#  Clean x-axis
import matplotlib.dates as mdates
ax6.xaxis.set_major_locator(mdates.MonthLocator(interval=6))
ax6.xaxis.set_major_formatter(mdates.DateFormatter('%Y-%m'))
ax6.xaxis.set_minor_locator(mdates.MonthLocator())

ax6.grid(which='major', linestyle='--', alpha=0.6)
ax6.grid(which='minor', linestyle=':', alpha=0.3)

plt.xticks(rotation=45)

# Category Trend
fig7, ax7 = plt.subplots(figsize=(10,5))

for col in category_trend.columns:
    ax7.plot(category_trend.index, category_trend[col], label=col)

ax7.legend()
ax7.set_title("Sales Trend by Category")
```

```
import matplotlib.dates as mdates
# Major ticks (big labels)
ax7.xaxis.set_major_locator(mdates.MonthLocator(interval=6))
ax7.xaxis.set_major_formatter(mdates.DateFormatter('%Y-%m'))

# Minor ticks (small lines every month)
ax7.xaxis.set_minor_locator(mdates.MonthLocator())

ax7.grid(which='major', linestyle='--', alpha=0.6)
ax7.grid(which='minor', linestyle=':', alpha=0.3)

# Rotate labels
plt.xticks(rotation=45)

#Forecast
fig8, ax8 = plt.subplots(figsize=(10,5))

# Actual data
ax8.plot(tech_sales.index, tech_sales, label='Actual', color='blue')

#  Fix bad starting drop
tech_forecast.iloc[0] = tech_sales.iloc[-1]

#  Combine for smooth transition
combined_index = tech_sales.index.append(tech_forecast.index)
combined_values = list(tech_sales.values) + list(tech_forecast.values)

ax8.plot(combined_index, combined_values, linestyle='--', color='red',
label='Forecast')

#  Confidence band
ax8.fill_between(
    tech_forecast.index,
    tech_forecast * 0.9,
    tech_forecast * 1.1,
    color='red',
    alpha=0.2
)
```

```
ax8.set_title("Technology Sales Forecast (ARIMA)")
ax8.legend()

# Convert to DataFrame
actual_df = tech_sales.reset_index()
actual_df.columns = ['Date', 'Actual Sales']
actual_df['Forecast'] = None

forecast_df = tech_forecast.reset_index()
forecast_df.columns = ['Date', 'Forecast']
forecast_df['Actual Sales'] = None

# Combine
final_df = pd.concat([actual_df, forecast_df], ignore_index=True)

# Sort by date
final_df = final_df.sort_values('Date')

# Major ticks (big labels)
ax8.xaxis.set_major_locator(mdates.MonthLocator(interval=6))
ax8.xaxis.set_major_formatter(mdates.DateFormatter('%Y-%m'))

# Minor ticks (small lines every month)
ax8.xaxis.set_minor_locator(mdates.MonthLocator())

ax8.grid(which='major', linestyle='--', alpha=0.6)
ax8.grid(which='minor', linestyle=':', alpha=0.3)

# Rotate labels
plt.xticks(rotation=45)

import matplotlib.dates as mdates

#SHIP MODE FIGURES

# fig9 — Sales
fig9, ax9 = plt.subplots(figsize=(8,4))
ax9.bar(ship_summary['Ship Mode'], ship_summary['Sales'])
ax9.set_title("Sales by Ship Mode")
```

```
# fig10 — Profit
fig10, ax10 = plt.subplots(figsize=(8,4))
ax10.bar(ship_summary['Ship Mode'], ship_summary['Profit'])
ax10.set_title("Profit by Ship Mode")
fig10.tight_layout()

# fig11 — Profit Margin
fig11, ax11 = plt.subplots(figsize=(8,4))
ax11.bar(ship_summary['Ship Mode'], ship_summary['Profit Margin'])
ax11.set_title("Profit Margin by Ship Mode")
fig11.tight_layout()

fig12, ax12 = plt.subplots(figsize=(10,5))
ax12.bar(top_states_sales['State'], top_states_sales['Sales'])
ax12.set_title("Top 10 States by Sales")
plt.xticks(rotation=45)

fig13, ax13 = plt.subplots(figsize=(10,5))
ax13.bar(top_states_profit['State'], top_states_profit['Profit'])
ax13.set_title("Top 10 States by Profit")
plt.xticks(rotation=45)

fig14, ax14 = plt.subplots(figsize=(10,5))
ax14.bar(loss_states['State'], loss_states['Profit'], color='red')  # already negative
ax14.set_title("Top Loss-Making States")
ax14.set_ylabel("Loss Amount")
plt.xticks(rotation=45)

st.title("Retail Business Dashboard")

# Category
st.header("Category Performance")
col1, col2 = st.columns(2)
with col1:
    st.pyplot(fig1)
with col2:
    st.pyplot(fig2)

best_cat = category_summary.sort_values(by='Profit', ascending=False).iloc[0]
```

```
st.subheader(" Best Performing Category")

st.success(f"{best_cat['Category']} is generating highest profit:
{best_cat['Profit']:.2f}")

with st.expander(" View Category Data"):
    st.dataframe(category_summary)

with st.expander(" View Profit Data"):
    st.dataframe(category_summary[['Category', 'Profit']])

# Risk
st.header("Risk Analysis")
st.pyplot(fig3)

worst = subcat_summary.iloc[0]
st.warning(f"{worst['Sub-Category']} is making lowest profit:
{worst['Profit']:.2f}")

st.subheader(" Recommended Action")

st.info(f"""
Focus on **{worst['Sub-Category']}**:
- Reduce heavy discounts
- Optimize pricing strategy
- If losses continue, consider removing or replacing this product line
""")

with st.expander(" View Sub-Category Data"):
    st.dataframe(subcat_summary)

# Region
st.header("Regional Insights")
col1, col2 = st.columns(2)
with col1:
    st.pyplot(fig4)
with col2:
    st.pyplot(fig5)

worst_region = region_summary.sort_values(by='Profit').iloc[0]
```

```
st.subheader("Weakest Region")
st.error(f"{worst_region['Region']} has lowest profit: {worst_region['Profit']:.2f}")

st.subheader("Business Recommendation")

if worst_region['Profit'] < 0:
    st.warning("Focus on improving performance in this region. Consider reducing
discounts or optimizing logistics.")
else:
    st.info("All regions are profitable. Focus on scaling high-performing regions.")

with st.expander("View Region Data"):
    st.dataframe(region_summary)

with st.expander(" View Profit Margin Data"):
    st.dataframe(region_summary[['Region', 'Profit Margin']])

# Trends
st.header("Trends")
col1, col2 = st.columns(2)
with col1:
    st.pyplot(fig6)
with col2:
    st.pyplot(fig7)

with st.expander(" View Monthly Sales Data"):
    st.dataframe(monthly_sales)

with st.expander("View Category Trend Data"):
    st.dataframe(category_trend)

#Technology Sales Forecast
st.header(" Forecast")

st.pyplot(fig8)

with st.expander(" View Forecast Data"):
    st.dataframe(final_df)
```

```
# Ship Mode Analysis
st.header(" Ship Mode Analysis")
st.pyplot(fig9)

col1, col2 = st.columns(2)
with col1:
    st.pyplot(fig10)
with col2:
    st.pyplot(fig11)

best_ship = ship_summary.sort_values(by='Profit', ascending=False).iloc[0]
st.success(f"{best_ship['Ship Mode']} is generating highest profit:
{best_ship['Profit']:.2f}")

with st.expander(" View Ship Mode Data"):
    st.dataframe(ship_summary)

#State Analysis
st.header(" State Analysis")
st.pyplot(fig12)
col1, col2, = st.columns(2)

with col1:
    st.pyplot(fig13)

with col2:
    st.pyplot(fig14)

worst_state = state_summary.sort_values(by='Profit').iloc[0]
best_state = state_summary.sort_values(by='Profit', ascending=False).iloc[0]
col1, col2 = st.columns(2)
with col1:
    st.success(f"""
 **Highest Profit**
**State:** {best_state['State']}
**Amount:** {best_state['Profit']:.2f}
""")
with col2:
    st.error(f"""
 **Highest Loss**
```

```
**State:** {worst_state['State']}
**Amount:** {worst_state['Profit']:.2f}
""")
with st.expander(" State Data Table"):
    st.dataframe(state_summary)
```

4.2.2 How to Run the Application:

To successfully execute the project, the following requirements and steps should be ensured before running the application:

Prerequisites:

- Ensure you are inside the project folder containing main.py, app.py, and the dataset file (e.g., superstore.csv).
- Python must be installed on your system (preferably Python 3.8 or above).
- Required libraries should be installed, including:
  - streamlit
  - pandas
  - numpy
  - matplotlib
  - statsmodels (for ARIMA forecasting)
- If not installed, install them using:

  pip install streamlit pandas numpy matplotlib statsmodels
- Make sure the dataset file is present in the correct directory.

Run Command:

After ensuring all the above requirements are satisfied, execute the following command in the terminal:

python -m streamlit run app.py

This will launch the application in your default web browser, where you can interact with the dashboard and perform analysis.

CHAPTER 5: RESULTS ANALYSIS AND VALIDATION

5.1. Output of main.py

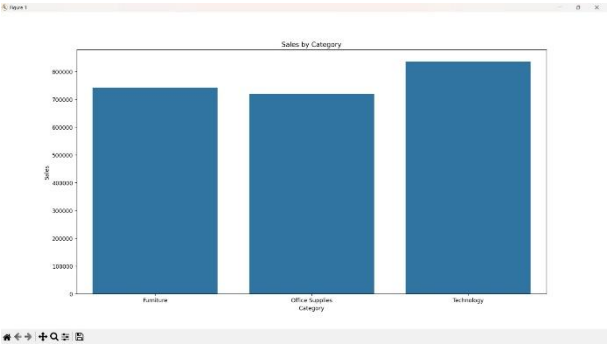


Figure5.1.1: Sales by Category

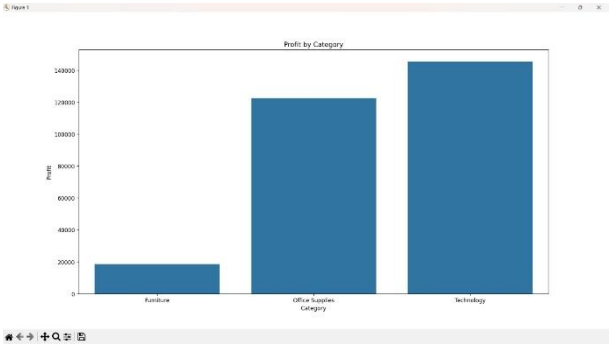


Figure5.1.2: Profit by Category

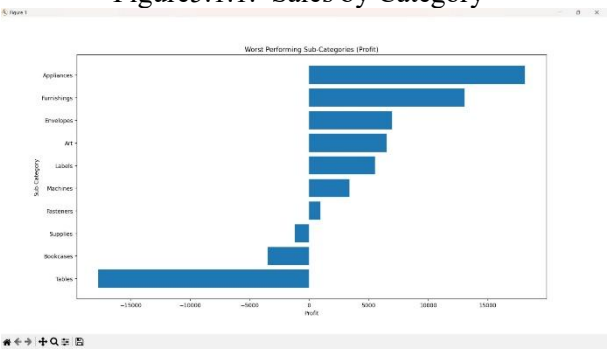


Figure5.1.3: Worst sub category

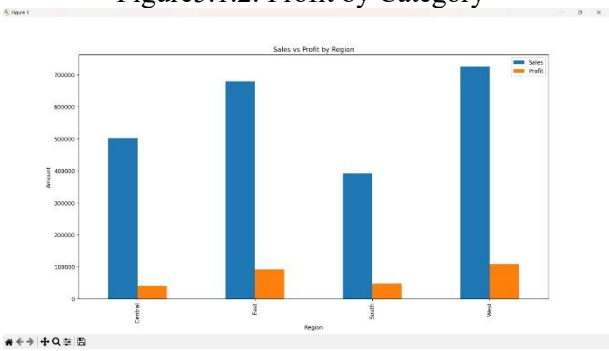


Figure5.1.4: Sales by region

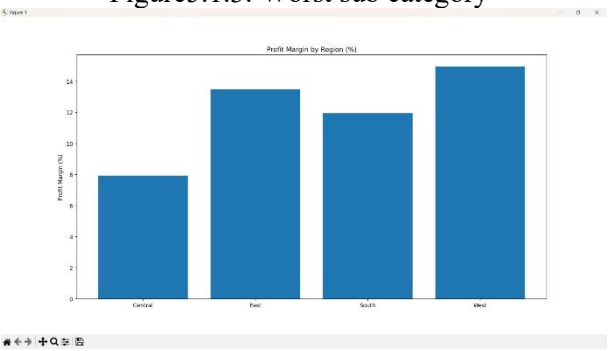


Figure5.1.5: Profit Margin Region

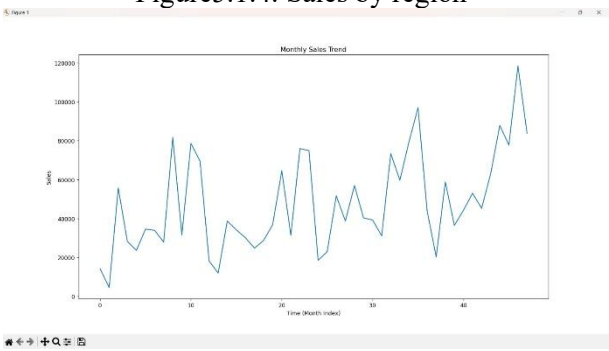


Figure5.1.6: Monthly Sales

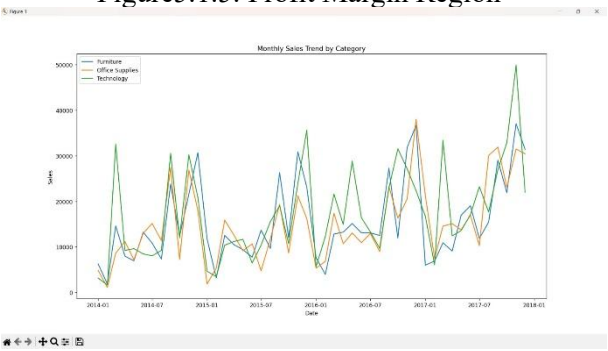


Figure5.1.7: Monthly Sales Trend

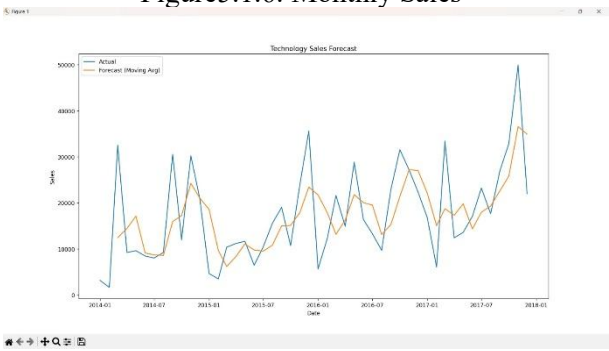


Figure5.1.8: Technology Sales Forecast

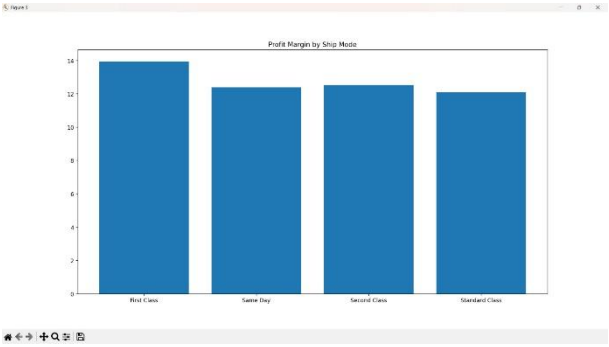


Figure5.1.9: Profit Margin by SM

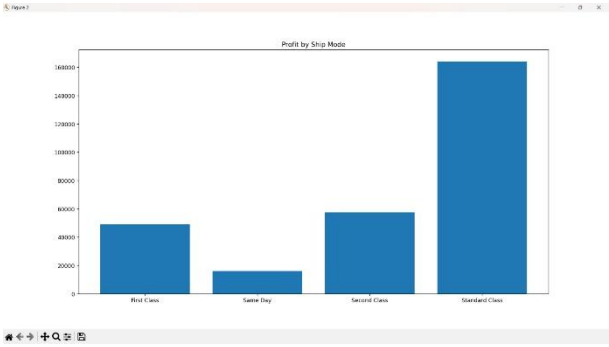


Figure5.1.10: Profit by SM

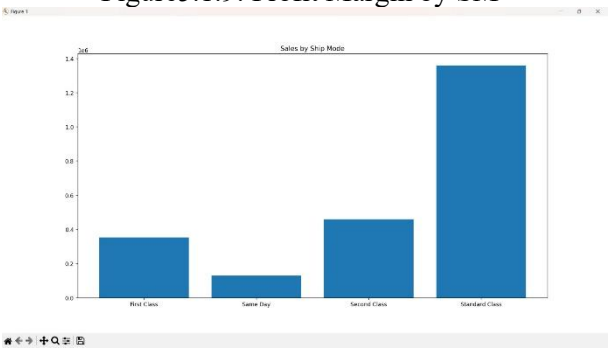


Figure5.1.11: Sales by SM

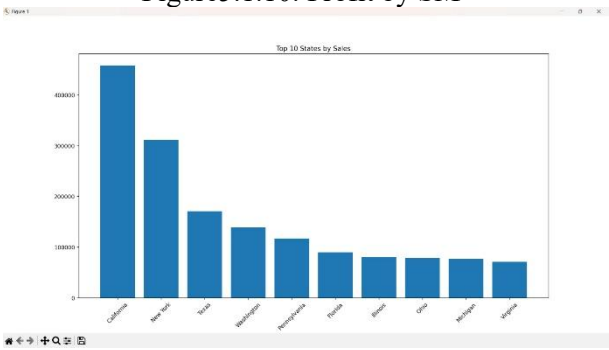


Figure5.1.12: States Sales

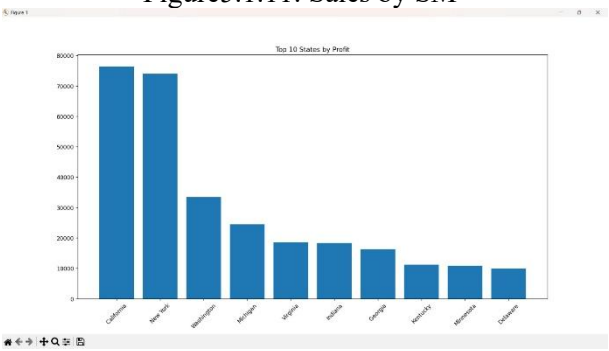


Figure5.1.15: Sales by Profit

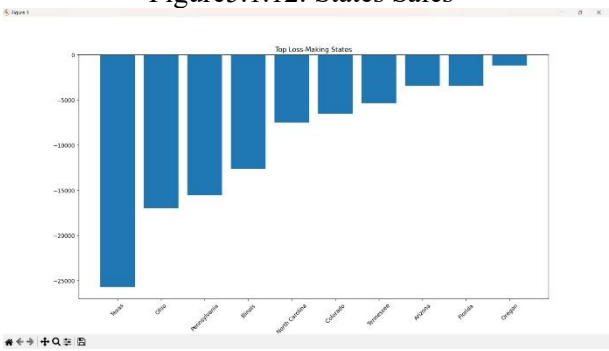


Figure5.1.14: Loss making Sales

5.2. Output of app.py

Figure5.2.1: Dashboard

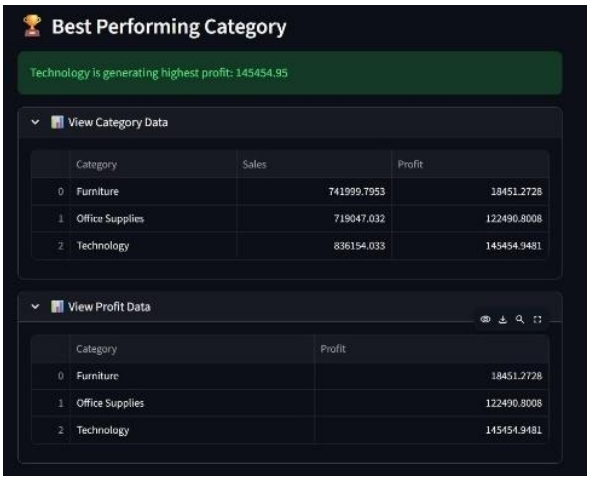


Figure5.2.2: Category Table

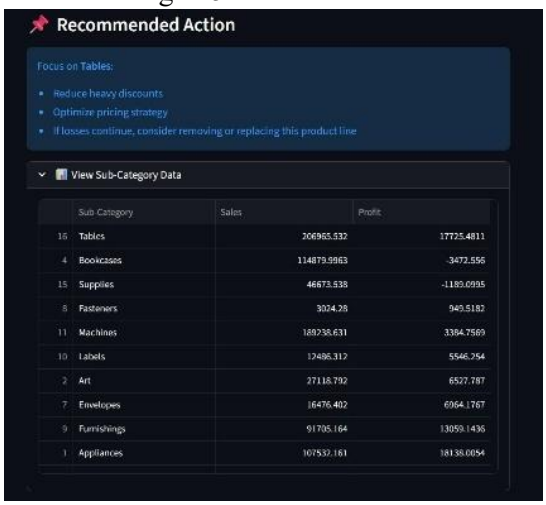


Figure5.2.3: Sub Category

Figure5.2.4: Region

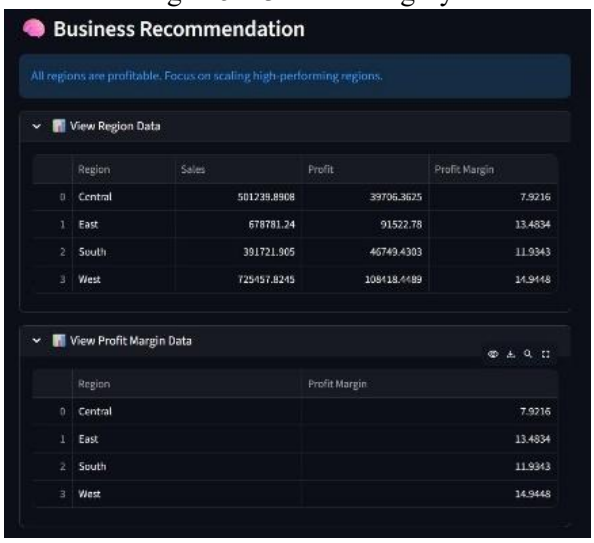


Figure5.2.5: Region Table

Figure5.2.6: Trends

View Monthly Sales Data

	Month-Year	Sales	Date
0	2014-01	14236.895	2014-01-01 00:00:00
1	2014-02	4519.892	2014-02-01 00:00:00
2	2014-03	55691.009	2014-03-01 00:00:00
3	2014-04	28295.345	2014-04-01 00:00:00
4	2014-05	23648.287	2014-05-01 00:00:00
5	2014-06	34595.1276	2014-06-01 00:00:00
6	2014-07	33946.393	2014-07-01 00:00:00
7	2014-08	27909.4685	2014-08-01 00:00:00
8	2014-09	81777.3508	2014-09-01 00:00:00
9	2014-10	31453.393	2014-10-01 00:00:00

Figure5.2.7: Monthly Sales

View Category Trend Data

Month-Year	Furniture	Office Supplies	Technology
2014-01-01 00:00:00	6242.525	4851.08	3143.29
2014-02-01 00:00:00	1839.658	1071.724	1608.51
2014-03-01 00:00:00	14573.956	8605.879	32511.174
2014-04-01 00:00:00	7944.837	11155.074	9195.434
2014-05-01 00:00:00	6912.787	7135.624	9599.876
2014-06-01 00:00:00	13206.1256	12953.037	8435.965
2014-07-01 00:00:00	10821.051	15121.208	8004.134
2014-08-01 00:00:00	7320.3465	11379.462	9209.66
2014-09-01 00:00:00	23816.4808	27423.298	30537.572
2014-10-01 00:00:00	12304.247	7211.128	11938.018

Figure5.2.8: Trend Data

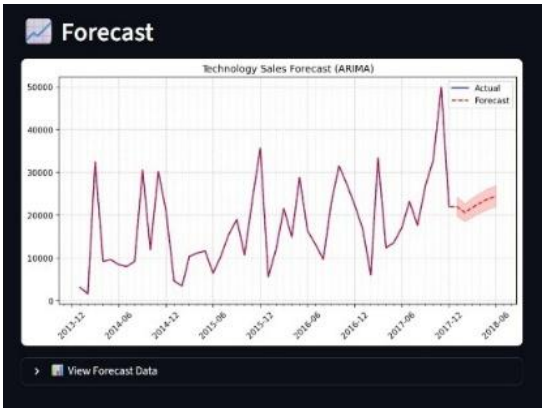


Figure5.2.9: ARIMA

View Forecast Data

	Date	Actual Sales	Forecast
0	2014-01-01 00:00:00	3143.29	None
1	2014-02-01 00:00:00	1608.51	None
2	2014-03-01 00:00:00	32511.174	None
3	2014-04-01 00:00:00	9195.434	None
4	2014-05-01 00:00:00	9599.876	None
5	2014-06-01 00:00:00	8435.965	None
6	2014-07-01 00:00:00	8004.134	None
7	2014-08-01 00:00:00	9209.66	None
8	2014-09-01 00:00:00	30537.572	None
9	2014-10-01 00:00:00	11938.018	None

Figure5.2.10: ARIMA Forecast

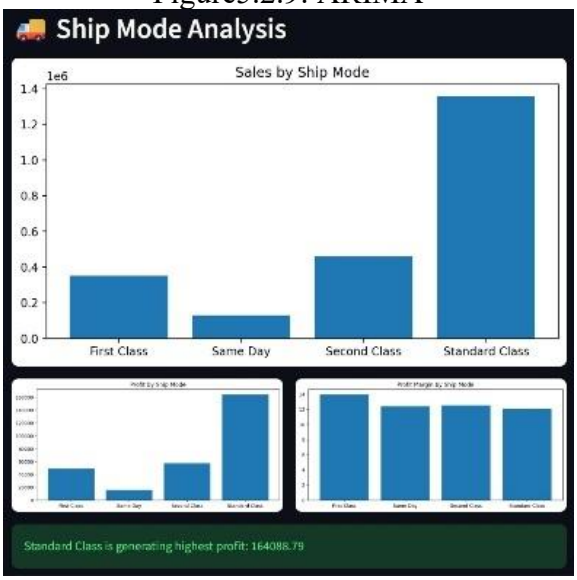


Figure5.2.11: SM Analysis

View Ship Mode Data

	Ship Mode	Sales	Profit	Profit Margin
0	First Class	351428.4229	48969.8399	13.9345
1	Same Day	128363.125	15891.7589	12.3803
2	Second Class	459193.5694	57446.6354	12.5103
3	Standard Class	1358215.743	164088.7875	12.0812

Figure5.2.12: SM Data

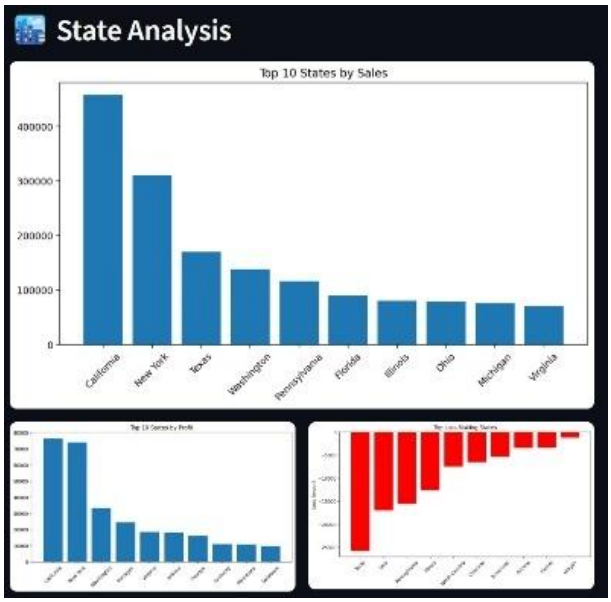
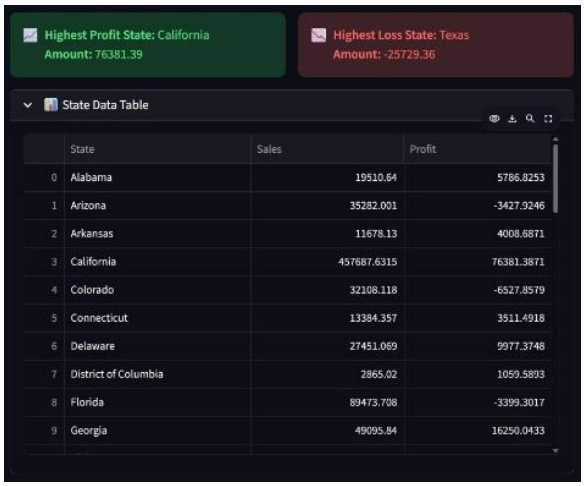
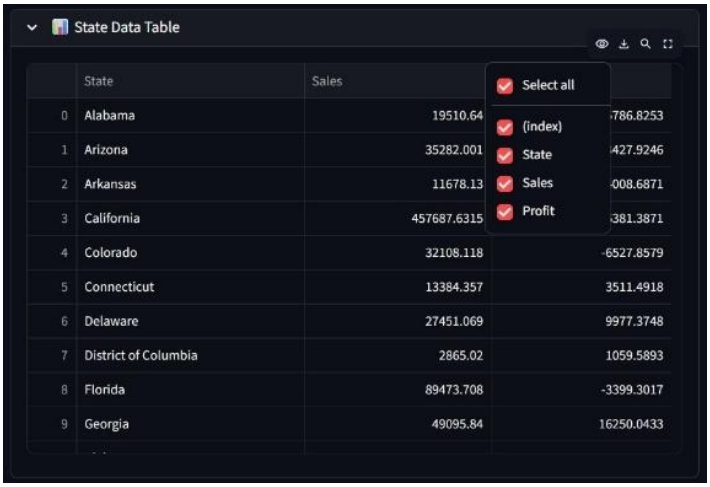


Figure5.2.13: State Sales



	State	Sales	Profit
0	Alabama	19510.64	5786.8253
1	Arizona	35282.001	-3427.9246
2	Arkansas	11678.13	4008.6871
3	California	457687.6315	76381.3871
4	Colorado	32108.118	-6527.8579
5	Connecticut	13384.357	3511.4918
6	Delaware	27451.069	9977.3748
7	District of Columbia	2865.02	1059.5893
8	Florida	89473.708	-3399.3017
9	Georgia	49095.84	16250.0433

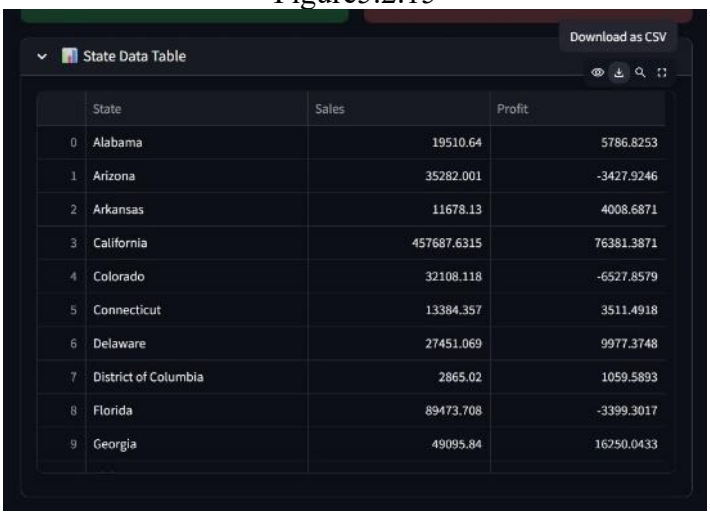
Figure5.2.14: State Data



	State	Sales	Profit
0	Alabama	19510.64	786.8253
1	Arizona	35282.001	427.9246
2	Arkansas	11678.13	008.6871
3	California	457687.6315	381.3871
4	Colorado	32108.118	-6527.8579
5	Connecticut	13384.357	3511.4918
6	Delaware	27451.069	9977.3748
7	District of Columbia	2865.02	1059.5893
8	Florida	89473.708	-3399.3017
9	Georgia	49095.84	16250.0433

Figure5.2.15

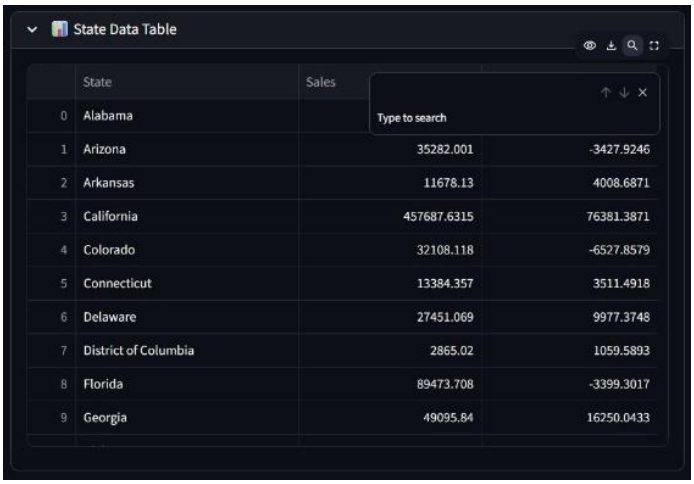
Allow user to select specific column in view



	State	Sales	Profit
0	Alabama	19510.64	5786.8253
1	Arizona	35282.001	-3427.9246
2	Arkansas	11678.13	4008.6871
3	California	457687.6315	76381.3871
4	Colorado	32108.118	-6527.8579
5	Connecticut	13384.357	3511.4918
6	Delaware	27451.069	9977.3748
7	District of Columbia	2865.02	1059.5893
8	Florida	89473.708	-3399.3017
9	Georgia	49095.84	16250.0433

Figure5.2.16

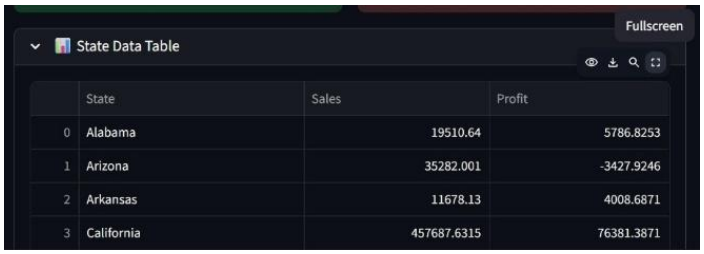
Allow download of specific tables as csv files



	State	Sales	
0	Alabama		
1	Arizona	35282.001	-3427.9246
2	Arkansas	11678.13	4008.6871
3	California	457687.6315	76381.3871
4	Colorado	32108.118	-6527.8579
5	Connecticut	13384.357	3511.4918
6	Delaware	27451.069	9977.3748
7	District of Columbia	2865.02	1059.5893
8	Florida	89473.708	-3399.3017
9	Georgia	49095.84	16250.0433

Figure5.2.17

Allow user to search for specific data from the table



	State	Sales	Profit
0	Alabama	19510.64	5786.8253
1	Arizona	35282.001	-3427.9246
2	Arkansas	11678.13	4008.6871
3	California	457687.6315	76381.3871

Figure5.2.18

Fullscreen mode

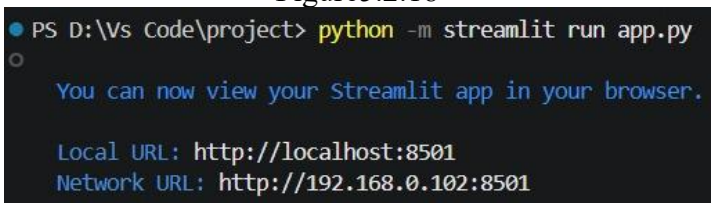


Figure5.2.19

Output screen of app.py

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1. Conclusion

The conclusion of this project summarizes the overall outcomes achieved through the design and development of the proposed system. The primary objective of the project was to create an efficient, reliable, and user-friendly solution that addresses the identified problem by improving data handling, reducing manual effort, and enhancing system performance. Based on the implementation and evaluation carried out during the project, it is expected that the system successfully fulfills these objectives. The developed solution demonstrates improved efficiency in processing tasks, provides accurate results through validation mechanisms, and offers a streamlined interface that enhances user interaction. These expected outcomes indicate that the system is capable of delivering a practical and effective solution in real-world scenarios.

In terms of expected results, the system was designed to minimize errors, reduce processing time, and improve overall productivity. The incorporation of automation and efficient algorithms allows the system to handle operations more effectively than traditional or earlier approaches. Additionally, the modular design ensures flexibility and scalability, enabling the system to adapt to future requirements. The expected outcome also includes enhanced usability, where users can interact with the system easily without requiring extensive technical knowledge. These results highlight the success of the project in achieving its intended goals and demonstrate the practical applicability of the developed system.

However, during the course of development and testing, certain deviations from the expected results may have been observed. These deviations could include minor performance limitations under specific conditions, such as handling extremely large datasets or operating in constrained environments. In some cases, the user interface may require further refinement to improve accessibility and responsiveness. Additionally, certain advanced features that were initially considered may not have been fully implemented due to constraints related to time, resources, or technical complexity. These deviations are not uncommon in software development projects and provide valuable insights for future improvements.

The reasons for these deviations can be attributed to various factors, including limited project scope, resource constraints, and the need to prioritize core functionalities over additional features. The decision to focus on essential features ensures that the system remains stable and reliable, even if it means excluding certain advanced capabilities. Furthermore, the complexity of integrating multiple functionalities within a limited timeframe can also contribute to minor inconsistencies in performance. These challenges highlight the importance of iterative development and continuous improvement in software engineering.

In conclusion, the project successfully achieves its primary objectives by delivering a functional and efficient system that addresses the identified problem. While there are some deviations from the expected results, they are minimal and provide opportunities for further enhancement. The overall outcome of the project demonstrates the effective application of software engineering principles and highlights the potential for future development and expansion.

6.2. Future work

The future work of this project outlines potential enhancements, modifications, and extensions that can be incorporated to improve the system's functionality, performance, and applicability. While the current implementation successfully addresses the core problem and meets the defined objectives, there remains scope for further development. These improvements can be undertaken in subsequent phases to transform the system into a more comprehensive and advanced solution capable of handling complex and large-scale requirements.

One of the primary areas for future enhancement is the expansion of system functionality to include advanced features and capabilities. This may involve integrating real-time data processing, enabling multi-user access, and incorporating advanced analytics for better decision-making. By extending the system to support multiple users simultaneously, the solution can be adapted for organizational or enterprise-level applications. Additionally, implementing more sophisticated data processing techniques and algorithms can further improve system efficiency and accuracy, particularly when dealing with large datasets.

Another important direction for future work is the improvement of the user interface and overall user experience. Although the current system provides a functional and user-friendly interface, there is always room for refinement. Enhancements such as responsive design, improved visualization of data, and interactive features can make the system more engaging and accessible. The incorporation of graphical dashboards and reporting tools can also provide users with better insights and facilitate easier interpretation of results.

The system can also be extended by integrating it with external platforms and technologies. For instance, connecting the system to cloud-based services can enhance scalability and allow remote access, making it more versatile and widely usable. Integration with databases, APIs, or third-party tools can further expand the system's capabilities and enable seamless data exchange.

From a technical perspective, future work may include optimizing the system for better performance and resource utilization. This involves refining algorithms, improving data structures, and enhancing overall system efficiency. Security can also be strengthened by implementing advanced authentication mechanisms, encryption techniques, and access control measures.

Furthermore, the project can explore alternative approaches and methodologies to improve the system's design and implementation. Continuous testing and feedback collection can also be used to identify areas for improvement and ensure that the system evolves according to user needs.

In conclusion, the future work section highlights the potential for extending and improving the current system beyond its initial implementation. By focusing on enhancements such as advanced features, improved usability, integration with modern technologies, and performance optimization, the system can be transformed into a more powerful and versatile solution, ensuring its relevance for future requirements.

REFERENCES:

1. McKinney, W. (2018). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. O'Reilly Media.
2. Hunter, J. D. (2007). "Matplotlib: A 2D Graphics Environment." *Computing in Science & Engineering*, 9(3), 90–95.
3. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research*, 12, 2825–2830.
4. Box, G. E. P., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). *Time Series Analysis: Forecasting and Control*. Wiley.
5. Hyndman, R. J., & Athanasopoulos, G. (2018). *Forecasting: Principles and Practice*. OTexts.
6. Streamlit Inc. (2023). *Streamlit Documentation*. Available at: https://docs.streamlit.io
7. Pandas Development Team. (2023). *Pandas Documentation*. Available at: https://pandas.pydata.org/docs/
8. NumPy Developers. (2023). *NumPy Documentation*. Available at: https://numpy.org/doc/
9. Statsmodels Developers. (2023). *Statsmodels Documentation*. Available at: https://www.statsmodels.org
10. Python Software Foundation. (2023). *Python Documentation*. Available at: https://docs.python.org/3/
11. Aggarwal, C. C. (2015). *Data Mining: The Textbook*. Springer.
12. Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques*. Elsevier.

Roles of Team Members

1. Sovit Ranjan – Implementation and testing
2. Avnish Kumar – Research and Documentation

Git repository Link:

https://github.com/sovit042/Retail-Analytics-Dashboard